बायेसियन और बृहत् भाषा मॉडल इंजनों का उपयोग करके आईओटी नेटवर्क के लिए ज़ीरो-ट्रस्ट प्रमाणीकरण और सतत विश्वास मूल्यांकन

Zero-Trust Authentication and Continuous Trust Evaluation for IoT Networks Using Bayesian and Large Language Model Engines

एक प्रतिवेदन

A Report

आवश्यकताओं की आंशिक पूर्ति में प्रस्तुत

Submitted in Partial Fulfilment of the Requirements

बी.टेक. डिग्री हेतु

for the Degree of

Bachelor of Technology

संगणक विज्ञान एवं अभियांत्रिकी में

Computer Science & Engineering

द्वारा

By

श्रेयश कुमार दुबे (२०२३३५६७)

स्पर्श सिंह (२०२३३२७७)

सिद्धेश चौबे (२०२३३५६९)

शुभम दया वाढेर (२०२३३५६८)

Shreyash Kumar Dube (20233567)

Sparsh Singh (20233277)

Siddhesh Choubey (20233569)

Shubham Daya Vadher (20233568)

पर्यवेक्षक: नीलम दयाल

Supervisor: Neelam Dayal



संगणक विज्ञान एवं अभियांत्रिकी विभाग

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

मोतीलाल नेहरू राष्ट्रीय प्रौद्योगिकी संस्थान इलाहाबाद

MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

प्रयागराज -२११००४, भारत

PRAYAGRAJ-211004, INDIA

नवम्बर २०२६

November, 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Motilal Nehru National Institute Of Technology Allahabad



CERTIFICATE

This is to certify that the report titled "**Zero-Trust Authentication and Continuous Trust Evaluation for IoT Networks Using Bayesian and Large Language Model Engines**" submitted by **Shreyash Kumar Dube (20233567)**, **Sparsh Singh (20233277)**, **Siddhesh Choubey (20233569)** and **Shubham Daya Vadher (20233568)** has been prepared with care under my guidance and supervision.

I hereby affirm that the material contained in this report is the actual product of the collaborative work done by the above-mentioned students. Throughout the project duration, they have demonstrated commendable dedication and adherence to academic integrity.

I further confirm that this report, in its entirety, has not been submitted to any other academic institution or organization for evaluation or publication.

Date: _____

_____

Neelam Dayal
Assistant Professor
Department of Computer Science and Engineering
Motilal Nehru National Institute of Technology, Allahabad

ACKNOWLEDGMENT

We wish to extend our thanks to everyone who helped us complete this project report titled **"Zero-Trust Authentication and Continuous Trust Evaluation for IoT Networks Using Bayesian and Large Language Model Engines"** and present it satisfactorily.

First of all, we express our sincere gratitude to our supervisor, **Neelam Dayal**, Assistant Professor in the Department of Computer Science and Engineering, for the steady support, patient guidance and mentorship given to us throughout this project. The questions raised during our reviews pushed us to measure what our system actually does instead of assuming it, and that habit shaped this whole report.

We are thankful to the Department of Computer Science and Engineering, MNNIT Allahabad, for giving us the facilities and the freedom to choose and pursue this topic. We also acknowledge the work of Yunfei Ge and Quanyan Zhu on game-theoretic zero-trust authentication (GAZETA), which gave us the starting point and the vocabulary for our design.

Lastly, we would like to thank each member of our team for their support and cooperation during the whole course of the project. The many evenings spent debugging trust curves together, and the honest disagreements about what the numbers meant, are what made this work come together.

Shreyash Kumar Dube (20233567)
Sparsh Singh (20233277)
Siddhesh Choubey (20233569)
Shubham Daya Vadher (20233568)

ABSTRACT

Internet of Things (IoT) devices are easy to compromise, and once an attacker holds a valid credential, a traditional perimeter network trusts every request that credential makes. Zero-trust security replaces this one-time trust with continuous verification: every request is authenticated and every device's trust is re-estimated from its behaviour. Our project builds on GAZETA, which treats trust as a probability updated by Bayes' rule from security evidence.

We designed and implemented **ZT-IoT Sentinel**, a zero-trust IoT simulator and access controller. Each device proves its identity on every request with a nonce-bound Schnorr-style zero-knowledge proof, and a Bayesian engine updates its trust in log-odds space from evidence the controller derives itself and from three imperfect sensors. A policy engine turns trust into ALLOW, STEP-UP (a fresh proof plus multi-factor authentication) or BLOCK with quarantine. As a second, independent engine we connected a large language model (LLM), run locally through Ollama or through the Google Gemini API, that receives only observed facts about a device and returns a trust score and an action as strictly validated JSON. The two engines run in fully isolated "twin worlds" that receive identical seeded traffic, so their decisions can be compared request by request against ground truth.

Over 210 seeded runs of seven scenarios, the Bayesian engine quarantined the attacker in every one of the 240 attacker runs, usually on the first malicious tick, while quarantining only 0.6–1.7% of honest devices, with a mean F1 between 0.956 and 0.995. The stealthy slow-and-low attacker took a median of 8.5 ticks (maximum 64) to catch. An ablation without sensor evidence shows the limits of our assumptions: attackers were still caught, but honest quarantines rose to 16–50%. The LLM comparison pipeline was validated with a clearly labelled rule-based stand-in, and one command repeats every experiment with a real model. The results show that a transparent Bayesian engine is a strong, explainable baseline that any LLM-based trust engine must be measured against.

CONTENTS

ACKNOWLEDGMENT i

ABSTRACT ii

LIST OF TABLES vi

LIST OF FIGURES viii

Chapter 1. INTRODUCTION 1

1.1 Problem Statement 1

1.2 Previous Approaches 2

1.3 Proposed Solution 3

1.4 Objectives 4

1.5 Contributions 5

1.6 Organisation of the Report 6

Chapter 2. RELATED WORK 7

2.1 Positioning of Our Work 11

Chapter 3. METHODOLOGY 12

3.1 System Overview 12

3.1.1 Simulation Clock 12

3.2 Simulated IoT Network and Scenarios 14

3.2.1 Devices and Topology 14

3.2.2 Agent Behaviour Models 14

3.2.3 Attack Scenarios 14

3.2.4 Sensor Model 15

3.3 Zero-Knowledge Device Authentication 16

3.4 Bayesian Trust Engine . . . 17
3.4.1 Log-Odds Formulation . . . 17
3.4.2 Evidence Derived by the Controller . . . 18
3.4.3 Likelihood Table and Tempering . . . 19
3.4.4 Worked Example . . . 20
3.4.5 How Sensor Alarms Are Generated . . . 21
3.4.6 Decay and Recovery . . . 21
3.5 Policy Engine . . . 22
3.5.1 Per-Request Processing Algorithm . . . 24
3.6 LLM Trust Engine . . . 25
3.6.1 Facts-Only Context . . . 25
3.6.2 Prompt and Output Schema . . . 26
3.6.3 Scheduling and Enforcement . . . 26
3.6.4 Strict Validation and Failure Handling . . . 27
3.6.5 Worked Evaluation . . . 28
3.6.6 Model Providers . . . 28
3.6.7 Rule-Based Stand-in for Pipeline Validation . . . 29
3.7 Twin-World Isolation and Reproducibility . . . 30
3.8 Evaluation Metrics . . . 31
3.9 Implementation . . . 32
3.9.1 Technology Stack . . . 32
3.9.2 Dashboard . . . 32
3.9.3 Testing . . . 38

Chapter 4. RESULTS **40**

4.1 Experimental Setup . . . 40
4.2 Detection Performance of the Bayesian Engine . . . 41
4.3 Trust Trajectories . . . 42
4.4 Sensor Evidence: Configured versus Observed . . . 43
4.5 Ablation: Behaviour-Only Evidence . . . 45
4.6 Sensitivity Analysis . . . 46
4.7 Bayesian versus LLM World (Stand-in) . . . 48

4.8 Trust Calibration 49
4.9 Comparison by Attack Behaviour 51
4.10 Case Study: One Multi-Attacker Run 53
4.11 Cost, Consistency and Explainability 53
4.12 Discussion 54
4.12.1 Credential Replay and Token Flooding 54
4.12.2 Scanning 54
4.12.3 Compromised Device with a Valid Token 55
4.12.4 Slow-and-Low Attacker 55
4.12.5 False Positives and Noisy Devices 55
4.12.6 Overall Analysis 56
4.12.7 How the Results Meet Our Objectives 56
4.12.8 Threats to Validity 56

Chapter 5. CONCLUSION **58**

5.1 Future Work 59

REFERENCES **61**

LIST OF TABLES

Table No.	Description	Page No.
2.1	Representative literature on zero-trust security, trust management and LLM-based security.	10
2.2	Comparison of approaches against the properties targeted by this project (✓ = yes, ○ = partly, – = no).	11
3.1	Devices, zones and configured links of the simulated IoT network.	14
3.2	Agent types and their behaviour.	15
3.3	The seven evaluation scenarios.	15
3.4	Evidence likelihoods and their weighted log-likelihood ratio (contribution to L).	19
3.5	Sensor rates and the contribution of an alarm or of silence to L (sensor weight 0.15).	19
3.6	Trust calculation for request 40-DEV-008-1 (compromised smart meter, seed 42), taken from the run log.	20
3.7	Sensor draws for request 40-DEV-008-1: an alarm fires when $u < p$.	21
3.8	Policy rules (first match wins).	23
3.9	Evaluation metrics.	31
3.10	Technology stack.	32
3.11	Simulation and engine parameters (`core/live_config.py`).	33
4.1	Bayesian engine results over 30 seeds per scenario.	41
4.2	Configured and observed sensor rates (Bayesian runs, all scenarios and seeds).	44

4.3 Ablation: Bayesian engine with and without sensor evidence (30 seeds per scenario). 45
4.4 One-at-a-time sensitivity analysis of the Bayesian engine (10 seeds per scenario per setting). 46
4.5 Twin-world comparison, part 1 (mean over 10 seeds; L = the rule-based LLM stand-in). 48
4.6 Twin-world comparison, part 2 (mean over 10 seeds; L = the rule-based LLM stand-in). 48
4.7 Reliability of trust values (all scenarios, 5 seeds; LLM world = the rule-based LLM stand-in). 50
4.8 Behaviour signatures used to recognise each attack kind from observed facts. 51
4.9 Detection by attack kind (attacker runs across all scenarios, 10 seeds; LLM world = the rule-based LLM stand-in). 51
4.10 Evidence each engine reacted to when it detected the attacker (number of runs in brackets). 52
4.11 Per-device comparison for the multi-attacker scenario, seed 0 (B = Bayesian, L = the rule-based LLM stand-in). 53
4.12 Objectives and the evidence that they were met. 56

LIST OF FIGURES

Figure No. **Description** **Page No.**

3.1 End-to-end flow of ZT-IoT Sentinel. Ground truth stays in the environment and is used only after the run for scoring. 13
3.2 The four phases of one simulation tick. 13
3.3 One zero-knowledge exchange with the teaching-size parameters $p = 227$, $q = 113$, $g = 3$. 17
3.4 Recovery towards the prior with $\lambda = 0.98$ when no new evidence arrives, computed by the real controller. 22
3.5 The seven stages of request 40-DEV-008-1 in the Bayesian world. 22
3.6 Log-odds and trust are linked by the sigmoid. The marked points come from a token-flood request in the project documentation ($L = 2.75 \rightarrow 0.47$, trust $0.94 \rightarrow 0.62$). 23
3.7 Trust bands and actions. The grey strips show the ± 0.03 hysteresis margin. 23
3.8 Device states and transitions. 24
3.9 How the LLM world produces and enforces a verdict. 25
3.10 Twin worlds: identical inputs, no shared state, and a comparison layer that only reads. 30
3.11 Live tab, Bayesian world only (compromised-device scenario after 42 ticks). 34
3.12 Live tab with both worlds running side by side. 35
3.13 Bayesian vs LLM tab: headline metrics and the attackers side by side. . 35
3.14 Results tab: detection metrics computed from ground truth after the run. 36
3.15 Trust calculation tab: the same request id shown in each world. 36

3.16 Attacks by behaviour: inferred attack kind and the evidence each engine reacted to. 37
3.17 Alarm system view: how the alarms for one request were drawn, and whole-run alarm statistics. 37
3.18 Classic mode: one request evaluated by both engines. 38

4.1 Ticks from attack start to quarantine for every attacker run (Bayesian engine, 30 seeds per scenario). 42
4.2 Variation across seeds: share of honest devices quarantined and device-level F1 for every run of every scenario (Bayesian engine). 43
4.3 Trust over time in the multi-attacker scenario (seed 0). Top: Bayesian world. Bottom: LLM world driven by the rule-based stand-in (step lines, because trust changes only at evaluations; gaps are PENDING). Shaded bands: Trusted (green) and Isolated (red). 44
4.4 Distribution of observed sensor rates per run; diamonds mark the configured rates. 44
4.5 Effect of removing sensor evidence on honest quarantines and on F1. . . 45
4.6 Sensitivity of honest quarantines (red, left axis) and mean F1 (blue, right axis) to four parameters. Dotted lines mark the defaults. 47
4.7 Per-request accuracy, Brier score and attacker requests allowed, per scenario (mean $\pm$ standard deviation over 10 seeds; LLM world = the rule-based LLM stand-in). 49
4.8 Reliability diagram: mean trust in each bin against the observed share of legitimate device-ticks (bins with at least 20 device-ticks). 50
4.9 Detection rate, median ticks to BLOCK and malicious requests allowed, per attack kind. 52

Chapter 1

INTRODUCTION

Smart homes, hospitals, factories and power grids now depend on thousands of small connected devices: temperature sensors, cameras, smart locks, meters, gateways and even medical infusion pumps. These devices are cheap, rarely patched and often run with default or long-lived credentials. When one of them is taken over, the real danger is not the device itself but what the attacker can reach from it. This project asks a simple question with a practical answer: *how should a network decide, request by request, whether to keep trusting a device?* We study it by building **ZT-IoT Sentinel**, a zero-trust access controller for a simulated IoT network. It authenticates every request with a zero-knowledge proof and continuously re-estimates each device's trust. It also compares two very different ways of doing that estimation: a transparent Bayesian engine and a large language model (LLM).

1.1 PROBLEM STATEMENT

Traditional network security draws a perimeter around the organisation and trusts what is inside it. A device that has once logged in, or that sits on the internal network, is assumed to be friendly. This assumption fails badly for IoT. Botnets such as Mirai showed how quickly weakly protected devices can be taken over at scale [1, 2], and surveys of IoT security report the same weaknesses again and again: weak authentication, insecure firmware and flat networks [3, 4]. Once inside, an attacker moves *laterally*. It uses a valid but stolen credential to reach devices it has no business talking to, probes ports, replays captured authentication messages, floods the controller with token requests, or quietly exfiltrates data [5].

The hardest case is the one GAZETA [6] was written for: an attacker that authenticates *correctly*. A compromised device that still holds its real secret passes every crypto-

graphic check, so authentication alone cannot stop it. Only its *behaviour* gives it away, and even then the evidence is noisy. Sensors raise false alarms, honest devices reboot and fail a proof or two, and maintenance staff occasionally access unusual targets. A good controller must therefore keep authenticating every request, keep weighing uncertain evidence, and still avoid locking out honest devices.

There is also a newer question. Large language models are increasingly proposed as security analysts that read logs and decide what is malicious [7, 8]. They are flexible and explain themselves in plain language, but they are also slow, non-deterministic and capable of confidently inventing facts [9]. Whether an LLM can replace a hand-built probabilistic trust engine is an open question, and answering it fairly needs both engines to see exactly the same traffic without either one learning from the other.

1.2 PREVIOUS APPROACHES

Historically, network access control relied on perimeter firewalls, VPNs and static access-control lists. They decide once, at the edge, and then trust whatever gets through. Intrusion detection systems (IDS) added signature- and anomaly-based alarms [10], but an alarm by itself does not tell the network what to do with the device that caused it, and every IDS has a false-positive rate that an operator must live with.

The *zero-trust* model, introduced by Kindervag [11] and standardised in NIST SP 800-207 [12], removes implicit trust altogether. Every request is authenticated and authorised on its own, by a policy engine that takes the requester's current trust into account. Surveys show many ways of building these components [13], and trust-management research for IoT has long modelled trust as a probability or reputation that is updated from evidence [14–17].

GAZETA [6] brings these ideas together for 5G IoT networks. It models the interaction between a defender and an agent of unknown identity as a Markov game with one-sided incomplete information. Trust is the defender's belief that the agent is legitimate, updated continuously with Bayes' rule from multiple sources of security evidence. The resulting equilibrium tells the defender how strongly to authenticate each request, which defends against lateral movement with stolen credentials. Related work by the same authors uses trust thresholds to make such policies explainable and adaptive [18, 19].

In parallel, LLMs built on the Transformer architecture [20, 21] and tuned to follow instructions [22] have been applied to log analysis and threat detection [7, 8]. Most of these studies, however, judge an LLM on a fixed dataset. They do not let the model take enforcement decisions in a running network, and they do not compare it under identical conditions with a principled Bayesian baseline.

Motivation: We chose this problem because it sits where three topics we enjoyed in our coursework meet: distributed systems, cryptography and machine learning. Papers like GAZETA describe the mathematics of continuous trust very well, but it is hard to *see* what Bayesian trust does on a live network, request after request, or to check whether a fashionable LLM would do better. We wanted a system that makes every trust calculation visible, that can be run again with the same seed and give the same answer, and that compares the two engines honestly.

1.3 PROPOSED SOLUTION

We propose **ZT-IoT Sentinel**, a zero-trust controller for a simulated IoT network together with a fair comparison of two trust engines. The design follows the zero-trust architecture of NIST SP 800-207 [12] and the trust model of GAZETA [6]. The system has the following parts.

- **A seeded IoT environment.** Nine devices in three security zones (Public IoT, Private IoT and a Critical Zone), connected by configured links, are driven by agents. Honest and "noisy-honest" agents behave normally, and five attacker types each model a lateral-movement technique: token flooding, credential replay, scanning, a compromised device with a valid credential, and a stealthy slow-and-low attacker. Every random choice comes from a stream keyed by (seed, device, tick, purpose, request index), so a run is exactly reproducible.
- **Zero-knowledge authentication on every request.** Each device proves knowledge of its secret with a Schnorr-style proof [23] made non-interactive with the Fiat–Shamir heuristic [24] and bound to a fresh nonce. A failed proof or a reused nonce blocks that request immediately.
- **A Bayesian trust engine.** Trust is held in log-odds form and updated after every re-

quest from evidence that the controller derives itself (no configured link, cross-zone access, sensitive target, rate spike, port or device scan, unusual payload, night-time access, replayed nonce, excess token requests) and from three imperfect sensors (IDS, SIEM and a behaviour monitor). Evidence weights temper correlated evidence, and a slow decay pulls trust back towards the prior.

- **A policy engine with step-up and quarantine.** Trust is mapped to three bands with hysteresis. Trusted devices are allowed. Suspicious devices must pass a step-up challenge (a fresh proof on a challenge nonce plus simulated multi-factor authentication). Isolated devices are quarantined for 30 ticks and then put on probation.
- **An LLM trust engine in an isolated twin world.** A second controller builds a facts-only summary of each device's recent behaviour and asks an LLM for a trust score and an action as JSON. It works with a local model through Ollama (default `llama3.2:3b`), with Google Gemini, or with any OpenAI-compatible server. The reply is validated strictly. A malformed reply is recorded as an error and never replaced with a Bayesian value.
- **A comparison and evaluation layer.** The two worlds receive identical traffic and never share state. After a run, ground truth is used only for scoring: detection rate, time to detect, false positives, per-request accuracy, Brier score, agreement, latency and consistency. A Streamlit dashboard shows everything live, including each trust calculation step by step, and a batch runner repeats every scenario over many seeds.

1.4 OBJECTIVES

The specific objectives of this project are:

1. **Build a reproducible zero-trust IoT testbed:** design a simulated IoT network with realistic honest, noisy and malicious device behaviour, where every run can be repeated exactly from its seed and ground truth never leaks to the controller [12].
2. **Authenticate every request without revealing secrets:** implement nonce-bound zero-knowledge proofs so that each request is cryptographically verified and replayed proofs are rejected [23, 25].
3. **Estimate trust continuously and transparently:** implement a Bayesian log-odds

trust engine, in the spirit of GAZETA, whose every update can be written out and checked by hand [6, 14].

4. **Enforce graded, reversible decisions:** map trust to ALLOW, STEP-UP or BLOCK with hysteresis, step-up challenges, time-limited quarantine and probation, so that a false alarm costs an honest device a challenge rather than permanent isolation.

5. **Integrate an LLM as an independent trust engine:** connect local and cloud LLMs through a facts-only interface with strict output validation and no fallback to the Bayesian engine [26, 27].

6. **Compare the engines fairly and measure honestly:** run both engines in isolated twin worlds on identical traffic, evaluate them over many seeds with clearly defined metrics, and test our own assumptions with an ablation study.

1.5 CONTRIBUTIONS

In completing these objectives, this project makes the following contributions:

- **A working zero-trust controller for IoT.** Every request is authenticated with a nonce-bound zero-knowledge proof and then judged by a continuously updated trust value. The policy engine has graded responses: ALLOW, STEP-UP and BLOCK with quarantine and probation.

- **A transparent Bayesian trust engine.** Each trust update is a sum of named log-likelihood terms, so any decision can be reproduced by hand from the log, as we do in Table 3.6. The engine follows the Bayesian view of trust used in GAZETA [6].

- **An LLM trust engine with strict guard-rails.** The language model sees only observed facts, must answer in a fixed JSON schema, is never silently replaced by the Bayesian engine, and has its stated reasons checked against the facts it received. It works with local models through Ollama, with Google Gemini, and with any OpenAI-compatible server.

- **A fair comparison method.** Two fully isolated twin worlds receive identical seeded traffic. A comparison layer scores both against ground truth only after the run, on detection, false positives, calibration, agreement, consistency, latency and explain-

ability.

- **An honest evaluation.** We report 210 seeded runs of the Bayesian engine, an ablation without sensor evidence, a sensitivity analysis of the main parameters and a trust-calibration study, and we state clearly which of our results depend on our own modelling assumptions.
- **A reproducible, inspectable tool.** A Streamlit dashboard shows every calculation live, a batch runner repeats every experiment from the command line, and 158 automated tests guard the system's correctness and isolation.

1.6 ORGANISATION OF THE REPORT

The rest of this report is organised as follows.

- **Chapter 2** reviews related work on zero-trust architecture, game-theoretic zero trust, trust management for IoT, zero-knowledge authentication and large language models in security, and positions our project against it.
- **Chapter 3** describes the methodology: the simulated IoT network and its attackers, zero-knowledge authentication, the Bayesian trust engine, the policy engine, the LLM trust engine, the twin-world isolation, the evaluation metrics and the implementation.
- **Chapter 4** presents and discusses the experimental results: detection performance, trust trajectories, sensor behaviour, the ablation and sensitivity studies, trust calibration and the comparison of the two worlds.
- **Chapter 5** concludes the report and outlines directions for future work.

Chapter 2

RELATED WORK

Our work draws on four lines of research: zero-trust architecture, trust management for IoT, zero-knowledge authentication, and the newer use of large language models in security. This chapter reviews each in turn and explains where our project fits.

Zero-trust architecture. The term "zero trust" was popularised by Kindervag [11], who argued that the trusted internal network was a dangerous fiction. Instead, every flow should be treated as untrusted until verified. NIST SP 800-207 [12] turned this idea into a reference architecture with three logical components: a *policy engine* that decides, a *policy administrator* that acts on the decision, and a *policy enforcement point* in the data path. A trust algorithm combines the identity of the requester with its current behaviour and other evidence. Syed et al. [13] survey the many ways of realising these components and point out that the trust algorithm is usually the least specified part. Our controller follows this structure directly. The Bayesian or LLM engine is the trust algorithm, the policy engine turns trust into an action, and the simulated network applies it.

Game-theoretic zero trust. Ge and Zhu [6] propose GAZETA for 5G IoT networks, where an attacker with stolen credentials moves laterally between devices. They model the defender and the agent as players in a Markov game in which the defender does not know the agent's true type. The defender's *trust* is its belief that the agent is legitimate, and it is refined with Bayes' rule as security evidence arrives. Equilibrium analysis then gives the authentication policy, and a moving-horizon method lets the defender make these decisions online. The same authors study explainable trust-threshold policies for enterprise networks [18] and discuss zero trust as a route to cyber resilience [19]. This builds on a wider body of game-theoretic models of advanced persistent threats [28]. We adopt GAZETA's view of trust as a Bayesian belief fed by several evidence sources, and

its idea that authentication strength should depend on trust (our step-up challenge). We do not solve the game itself. Instead we use fixed, explainable thresholds, which makes every decision easy to audit.

Trust and reputation for IoT. Long before zero trust, reputation systems estimated how much to trust a party from its past behaviour. The beta reputation system [14] updates a probability of good behaviour from counts of positive and negative evidence, and later surveys catalogue many variants [15]. For IoT specifically, Yan et al. [16] and Sicari et al. [17] identify trust management as a core open problem, because devices are numerous and heterogeneous and behave in noisy ways. Our log-odds update is a simple, transparent member of this family, with explicit likelihood ratios for each kind of evidence.

IoT threats and lateral movement. The Mirai botnet [1, 2] showed how quickly weakly secured IoT devices can be taken over, and how one compromised device becomes a launching point for others [29]. Large surveys document the same weaknesses across device classes [3, 4]. MITRE ATT&CK [5] catalogues lateral-movement techniques such as the use of valid accounts and remote services, and Bowman et al. [30] detect lateral movement from authentication graphs. Intrusion detection systems [10] supply alarms but also false positives. In our design, IDS, SIEM and behaviour-monitor alarms are therefore only weighted pieces of evidence, never verdicts.

Zero-knowledge authentication. Zero-knowledge proofs, introduced by Goldwasser, Micali and Rackoff [25], let a prover convince a verifier that it knows a secret without revealing anything about it. Schnorr's identification scheme [23] is the classic efficient example, and the Fiat–Shamir heuristic [24] makes it non-interactive by deriving the challenge from a hash. Binding the hash to a fresh nonce makes every proof single-use, so a replayed proof can be recognised. Our devices authenticate this way on every request.

Large language models in security. Transformer-based language models [20] trained at scale show strong few-shot reasoning [21], and instruction tuning [22] makes them follow structured prompts. Open models such as Llama 3 [26] can run locally, while hosted models such as Gemini [27] offer more capacity through an API. A recent systematic review [7] finds LLMs applied to vulnerability detection, malware analysis, log analysis and intrusion detection, and Ferrag et al. [8] build a lightweight language model for IoT threat detection. The same literature warns that LLM outputs can be unfaithful to their inputs [9]. This is why our LLM engine receives only observed facts, must answer

in a fixed JSON schema, and has its stated factors checked against the facts it was given.

Research gap. To our knowledge, the studies above evaluate either a probabilistic trust model or an LLM, but rarely both under identical conditions, and rarely with the engine actually enforcing its decisions in a running network. Our project contributes a reproducible testbed in which a Bayesian engine and an LLM engine make live zero-trust decisions on identical traffic in isolated worlds, and are then scored with the same metrics against ground truth. Table 2.1 summarises the key works.

Table 2.1: Representative literature on zero-trust security, trust management and LLM-based security.

Author(s)	Year	Setting	Method	Key findings / contribution
Ge and Zhu	2024	5G IoT networks	Markov game with incomplete information, Bayesian trust	GAZETA: trust-dependent authentication policy against lateral movement with stolen credentials [6].
Ge and Zhu	2022	Enterprise networks	Trust-threshold policy	Explainable and adaptive zero-trust defence based on trust thresholds [18].
Rose et al. (NIST)	2020	General	Reference architecture	Defines the policy engine, administrator and enforcement point of a zero-trust architecture [12].
Kindervag	2010	Enterprise networks	Security model	Introduces the zero-trust model: no implicit trust inside the perimeter [11].
Syed et al.	2022	Survey	Literature review	Surveys zero-trust components; notes that trust algorithms are under-specified [13].
Jøsang and Ismail	2002	Online services	Beta reputation	Probability of good behaviour updated from positive and negative evidence [14].
Yan et al.	2014	IoT	Survey	Identifies trust management as a core requirement for IoT [16].
Antonakakis et al.	2017	Internet-wide IoT	Measurement study	Shows how the Mirai botnet compromised IoT devices at scale [1].
Bowman et al.	2020	Enterprise networks	Unsupervised graph learning	Detects lateral movement from authentication graphs [30].
Khraisat et al.	2019	Networks	Survey	Reviews IDS techniques, datasets and their false-positive problem [10].
Schnorr	1991	Smart cards	Zero-knowledge identification	Efficient proof of knowledge of a discrete logarithm [23].
Xu et al.	2024	Cyber security	Systematic review	Maps LLM applications in security and their open challenges [7].
Ferrag et al.	2023	IoT / IIoT	Lightweight language model	Language model for IoT cyber-threat detection [8].
Ji et al.	2023	Natural language generation	Survey	Documents hallucination: outputs unfaithful to the input [9].

2.1 POSITIONING OF OUR WORK

Table 2.2 compares the main families of approaches discussed above with our project, on the properties that matter most for stopping lateral movement in an IoT network. A perimeter firewall authenticates once and then trusts the device, so a stolen credential defeats it. An IDS on its own raises alarms but does not decide what happens to the device, and its false positives must be handled by people. Reputation systems estimate trust from behaviour, but they are usually not tied to per-request authentication. The NIST architecture [12] defines the right components but leaves the trust algorithm open. GAZETA [6] supplies a principled Bayesian trust model and links authentication strength to trust. LLM-based analysts [7] are flexible and explain themselves in natural language, but they are rarely allowed to enforce their own decisions, and they can produce unfaithful explanations [9].

Table 2.2: Comparison of approaches against the properties targeted by this project (✓ = yes, ○ = partly, – = no).

Approach	**Auth. on every request**	**Behaviour-based trust**	**Catches valid-credential attacker**	**Graded response**	**Every decision explainable**	**Two engines compared on identical traffic**
Perimeter firewall / VPN	–	–	–	–	✓	–
IDS alarms alone [10]	–	○	○	–	○	–
Reputation systems [14]	–	✓	○	–	✓	–
NIST zero trust [12]	✓	○	○	○	○	–
GAZETA [6]	✓	✓	✓	✓	○	–
LLM security analyst [7]	–	✓	○	–	○	–
This project	✓	✓	✓	✓	✓ (Bayesian)	✓

Our project is therefore not a new trust theory. It builds on GAZETA's model and makes three practical additions. First, it is a complete, runnable system in which every request is authenticated with a zero-knowledge proof and every trust update can be inspected. Second, it treats an LLM as a full, enforcing trust engine rather than an advisor, with strict guard-rails. Third, it evaluates both engines in a way that is fair by construction: identical inputs, no shared state, the same metrics, and ground truth used only after the run.

Chapter 3

METHODOLOGY

This chapter explains how we built the system. It first gives the overall architecture, then describes the simulated IoT network and its attackers, the zero-knowledge authentication, the Bayesian trust engine, the policy engine and the LLM trust engine. It ends with how the two engines are kept isolated, how we measure them, and how the software is implemented and tested. Every number in this chapter is taken from the project's configuration file, `core/live_config.py`. These numbers are modelling assumptions chosen and justified by us, not values measured from real IoT traffic.

3.1 SYSTEM OVERVIEW

ZT-IoT Sentinel is split into an *environment*, which knows the ground truth, and a *controller*, which does not (Figure 3.1). The environment holds the devices, their links and the agents that drive them. An agent decides what its device does on each tick, for example sending telemetry to a gateway, requesting a token, or, for an attacker, probing an unlinked device. Every action passes through a *network tap* that strips all ground-truth fields and hands the controller an `ObservedEvent`. This event holds only what a real controller could see: the source and destination, the request kind, port, payload size, zones, target sensitivity, nonce and zero-knowledge proof. Creating an observed event that contains a ground-truth key raises an error, and an automated test checks that the controller module never imports the agents, the sensors or the simulator.

3.1.1 Simulation Clock

Time advances in discrete *ticks*. One simulated day is 100 ticks, and ticks 80–99 of each day are treated as night. Each tick has four phases (Figure 3.2). First, `begin_tick`

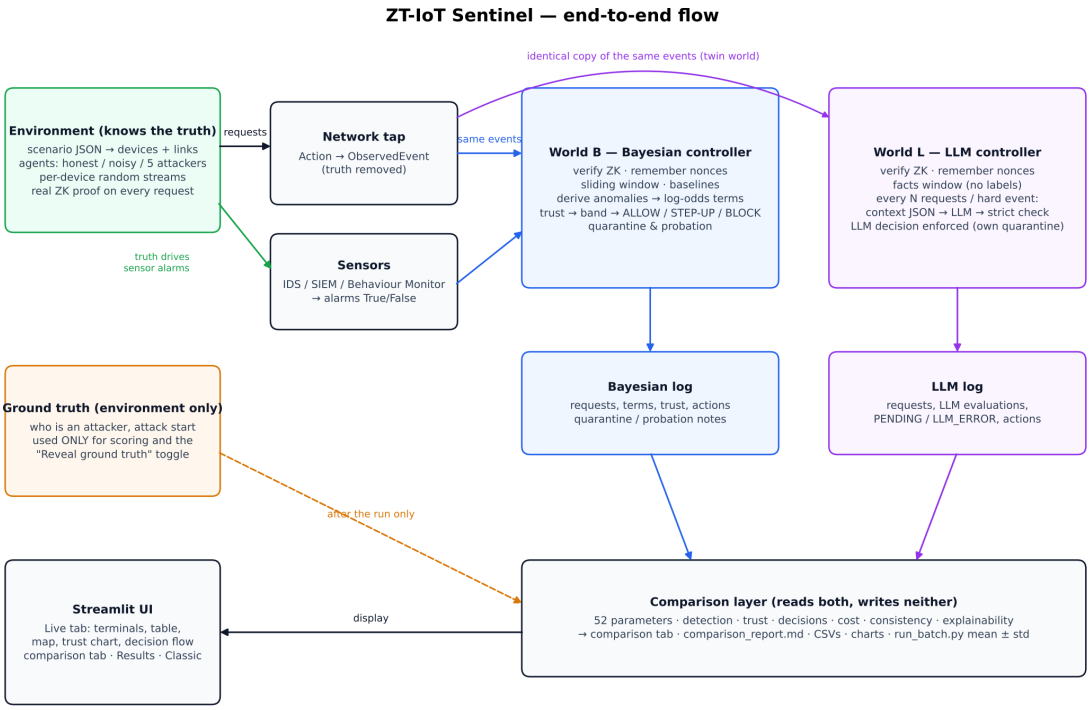


Figure 3.1: End-to-end flow of ZT-IoT Sentinel. Ground truth stays in the environment and is used only after the run for scoring.

releases devices whose quarantine has ended, applies trust decay and, after the warm-up period, freezes each device's learned baseline. Next, every device's requests are generated and processed one by one. Then `end_tick` lets the LLM world evaluate the devices that are due. Finally, trust and state are recorded for the charts. Detection times in this report are counted in ticks, which is fair to both engines. Wall-clock latency is reported separately.

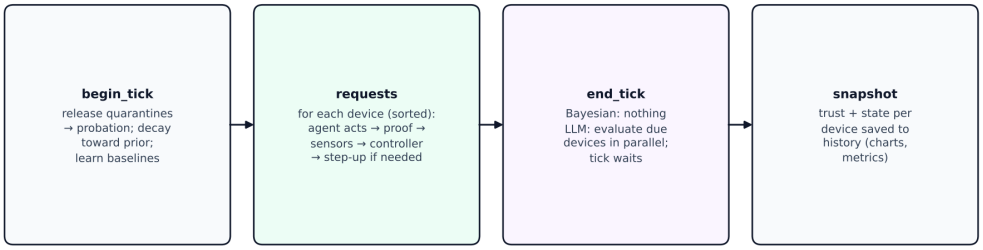


Figure 3.2: The four phases of one simulation tick.

3.2 SIMULATED IOT NETWORK AND SCENARIOS

3.2.1 Devices and Topology

All scenarios use the same network of 9 devices in three zones, joined by 8 configured directed links (Table 3.1). Each device has a *target sensitivity* between 0 and 1. Targets at 0.75 or above, such as the industrial actuator and the infusion pump, count as sensitive. The configured links are the controller's model of legitimate traffic: a request along a link is ordinary, while a request with no link behind it is evidence of lateral movement.

Table 3.1: Devices, zones and configured links of the simulated IoT network.

ID	Device	Zone	Protocol	Sensitivity	Configured links to
DEV-001	Temperature Sensor	Public IoT	MQTT	0.25	DEV-003
DEV-002	Smart Camera	Public IoT	MQTT	0.45	DEV-003
DEV-003	Gateway Controller	Private IoT	HTTPS	0.65	DEV-004
DEV-004	Industrial Actuator	Critical Zone	CoAP	0.95	—
DEV-006	Smart Lock	Private IoT	BLE	0.75	DEV-010
DEV-007	Smart Thermostat	Public IoT	MQTT	0.50	DEV-010
DEV-008	Smart Meter	Private IoT	LoRaWAN	0.75	DEV-010
DEV-009	Infusion Pump	Critical Zone	HTTPS	0.95	—
DEV-010	Edge Gateway	Private IoT	HTTPS	0.75	DEV-003, DEV-009

3.2.2 Agent Behaviour Models

Each device is driven by one agent type (Table 3.2). Honest agents send 0–2 ordinary requests per tick along their links or to the controller and ask for a token about once every 15 ticks. Noisy-honest agents add legitimate but suspicious-looking events: firmware updates, reboots followed by one or two damaged proofs, and occasional maintenance access to an unlinked device in another zone. These devices exist to test whether the controller can tell "odd" from "malicious". All attackers stay dormant and behave honestly until their `attack_start` tick, so that the controller learns a clean baseline first, and they keep sending honest cover traffic after they activate.

3.2.3 Attack Scenarios

Seven scenarios combine these agents (Table 3.3). *Normal day* contains no attacker and measures false positives. Five scenarios each contain one attacker type, and *multi-attacker* runs three attackers that start at different times. Every scenario also contains

Table 3.2: Agent types and their behaviour.

Agent type	Behaviour after activation
Honest	0–2 ordinary requests per tick along configured links; a token request about every 15 ticks; only telemetry at night.
Noisy-honest	Honest, plus firmware updates (2% of ticks), reboots (1%, followed by 1–2 failed proofs) and maintenance access to an unlinked device in another zone (1.5%).
Token flood	4–7 token requests per tick.
Credential replay	2–3 resent captured proofs per tick, either its own (nonce reuse) or another device's (wrong key).
Scanning	3–6 probes per tick to random devices on commonly attacked ports (22, 23, 80, 443, 502, 1883, 3389, 5683, 8080, 8443).
Compromised (valid token)	Holds its real secret, so every proof passes; 1–2 malicious moves per tick: lateral moves, access to sensitive targets, or exfiltration with payloads about 12× normal size.
Slow-and-low	The same moves as the compromised agent, but only on about 15% of ticks, hidden in normal traffic.

noisy-honest devices, so false alarms are always possible.

Table 3.3: The seven evaluation scenarios.

Scenario	Ticks	Special agents (all other devices are honest)
Normal day	160	no attacker; noisy-honest: DEV-002, DEV-007, DEV-008, DEV-010
Compromised (valid token)	160	DEV-008 compromised (valid token) from t = 40; noisy-honest: DEV-002, DEV-007, DEV-010
Credential replay	160	DEV-006 credential replay from t = 40; noisy-honest: DEV-002, DEV-007, DEV-008, DEV-010
Scanning	160	DEV-002 scanning from t = 40; noisy-honest: DEV-007, DEV-008, DEV-010
Token flood	160	DEV-007 token flood from t = 40; noisy-honest: DEV-002, DEV-008, DEV-010
Slow-and-low	250	DEV-006 slow-and-low from t = 40; noisy-honest: DEV-002, DEV-007, DEV-008, DEV-010
Multi-attacker	200	DEV-002 scanning from t = 35; DEV-007 token flood from t = 90; DEV-008 compromised (valid token) from t = 60; noisy-honest: DEV-010

3.2.4 Sensor Model

Three simulated sensors watch every request: an intrusion detection system (IDS), a security information and event management system (SIEM) and a behaviour monitor (BM). Each one raises an alarm on a malicious request with its true-positive rate and on a legitimate request with its false-positive rate (Table 3.5). The sensors are driven by ground truth inside the environment. The controller only sees whether each alarm fired, never why.

Treating sensors this way is an optimistic assumption, and Section 4.5 measures how much our results depend on it.

3.3 ZERO-KNOWLEDGE DEVICE AUTHENTICATION

Every request carries a Schnorr-style proof of knowledge of the device's secret x [23], made non-interactive with the Fiat–Shamir heuristic [24]. The public parameters are a prime $p = 227$, a prime $q = 113$ dividing $p - 1$, and a generator $g = 3$ of the order-q subgroup. At enrolment the controller stores only the public key $y = g^x \bmod p$. For each request, the device picks a random r and computes

$$t = g^r \bmod p, \tag{3.1}$$

$$c = H(id \,\|\, nonce \,\|\, t) \bmod q, \tag{3.2}$$

$$s = (r + c\,x) \bmod q, \tag{3.3}$$

where H is SHA-256 and the nonce is fresh for every request (for example `DEV-001-40-0`). The controller recomputes c and accepts only if

$$g^s \equiv t \cdot y^c \pmod p. \tag{3.4}$$

This holds because $g^s = g^{r+cx} = t \cdot y^c$. Because the nonce is part of the hash, a proof is valid for one request only. The controller remembers every nonce it has seen from each device, so a replayed proof is detected even if its arithmetic is correct. A failed proof or a reused nonce is a *hard gate*: that request is blocked immediately, whatever the device's trust. Figure 3.3 shows one complete exchange with real numbers.

> The parameters above are deliberately tiny so that every step can be checked by hand in a report. They are not secure; a real deployment would use a standard elliptic-curve group.

A valid proof shows that the requester knows the secret. It does not show that the requester is behaving well, and a compromised device with its real secret passes every proof. This is exactly the case GAZETA addresses [6], and it is why authentication is

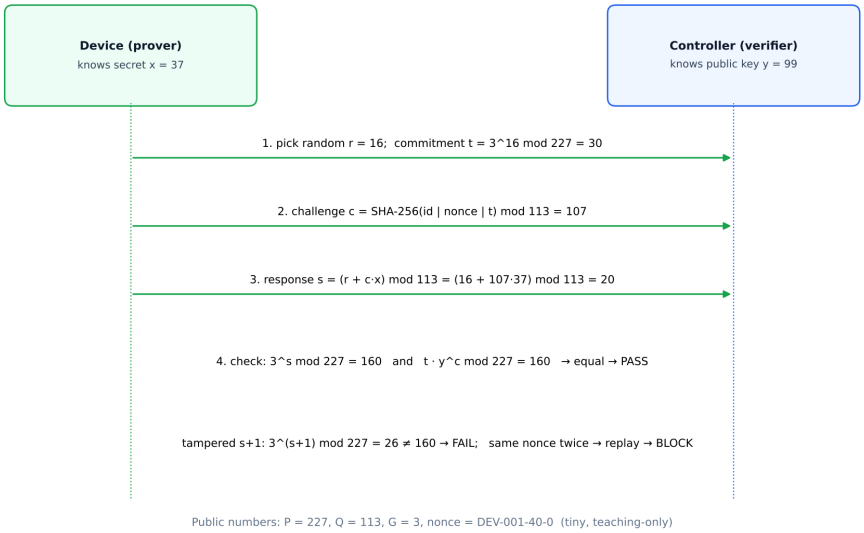


Figure 3.3: One zero-knowledge exchange with the teaching-size parameters $p = 227$, $q = 113$, $g = 3$.

followed by trust evaluation.

3.4 BAYESIAN TRUST ENGINE

3.4.1 Log-Odds Formulation

For each device the controller keeps a trust value $T \in [0.01, 0.99]$, read as the probability that the device is currently legitimate. Internally it works with the log-odds

$$L = \text{logit}(T) = \ln \frac{T}{1-T}, \qquad T = \sigma(L) = \frac{1}{1+e^{-L}}, \tag{3.5}$$

because Bayes' rule becomes a simple sum in this form. Every enrolled device starts from the same prior $\pi = 0.8$, since the controller has no per-device knowledge. Once per tick, trust decays towards the prior,

$$L \leftarrow \lambda L + (1-\lambda)\,\text{logit}(\pi), \qquad \lambda = 0.98, \tag{3.6}$$

so that old evidence slowly loses weight and a device that behaves well can recover. For every processed request, the evidence is added:

$$L \leftarrow L + \sum_{e \in E} w_e \ln \frac{P(e \mid \text{legit})}{P(e \mid \text{mal})} + \sum_{s \in S} w_s \ln \frac{P(o_s \mid \text{legit})}{P(o_s \mid \text{mal})}, \tag{3.7}$$

where E is the set of evidence events derived for the request, o_s is the observation of sensor s (alarm or silence), $w_e = 0.5$ and $w_s = 0.15$. For an alarm, $P(o_s \mid \text{legit})$ is the sensor's false-positive rate and $P(o_s \mid \text{mal})$ its true-positive rate. For silence, these are one minus each rate. Finally $T = \min(\max(\sigma(L), 0.01), 0.99)$. The floor and ceiling keep trust away from certainty, so recovery is always possible.

3.4.2 Evidence Derived by the Controller

The anomaly events are not labels sent by the environment. The controller derives them itself from what it has observed, using a 20-tick sliding window per device, the device's own baseline (request rate and mean payload per request kind, learned in its first 20 ticks), the configured topology and the clock:

- **no_topology_edge**: there is no configured link from source to destination;
- **cross_zone_access**: the target's zone is not reached by any configured link of the source;
- **sensitive_target_access**: the target's sensitivity is at least 0.75 and no link covers it;
- **token_request**: token requests beyond the free allowance of 3 per window;
- **rate_spike**: the window count exceeds both $3\times$ the baseline and the baseline plus 20;
- **port_scan / device_scan**: more than 5 distinct ports or more than 4 distinct destinations in the window;
- **unusual_payload**: the payload is more than $4\times$ the baseline mean for that request kind;
- **off_hours_access**: a service or token request at night;

- **zk_auth_fail** and **token_replay**: from the authentication step.

3.4.3 Likelihood Table and Tempering

Table 3.4 lists every evidence type with the two likelihoods we assigned and the resulting contribution to L. Ordinary request kinds give a small positive term, because attackers also send ordinary traffic. Rare events such as a replayed nonce give a large negative term. Table 3.5 gives the same for the sensors.

Table 3.4: Evidence likelihoods and their weighted log-likelihood ratio (contribution to L).

Evidence e	$P(e \mid$ **legit**$)$	$P(e \mid$ **mal**$)$	ln **ratio**	$\times\, w_e = 0.5$
normal_telemetry	0.98	0.9	$+0.09$	$+0.04$
authorized_service	0.97	0.9	$+0.07$	$+0.04$
read_sensor	0.96	0.9	$+0.06$	$+0.03$
firmware_update	0.9	0.95	-0.05	-0.03
reboot	0.85	0.9	-0.06	-0.03
token_request	0.1	0.7	-1.95	-0.97
zk_auth_fail	0.03	0.4	-2.59	-1.30
token_replay	0.005	0.5	-4.61	-2.30
rate_spike	0.03	0.5	-2.81	-1.41
port_scan	0.01	0.4	-3.69	-1.84
device_scan	0.01	0.4	-3.69	-1.84
cross_zone_access	0.05	0.4	-2.08	-1.04
sensitive_target_access	0.05	0.5	-2.30	-1.15
no_topology_edge	0.02	0.5	-3.22	-1.61
unusual_payload	0.03	0.4	-2.59	-1.30
off_hours_access	0.1	0.4	-1.39	-0.69
step_up_pass	0.97	0.15	$+1.87$	$+0.93$
step_up_fail	0.03	0.85	-3.34	-1.67

Table 3.5: Sensor rates and the contribution of an alarm or of silence to L (sensor weight 0.15).

Sensor	**TP rate**	**FP rate**	**Alarm term**	**Silence term**
IDS	0.92	0.04	-0.47	$+0.37$
SIEM	0.88	0.06	-0.40	$+0.31$
Behaviour Monitor (BM)	0.9	0.05	-0.43	$+0.34$

The weights w_e and w_s are a deliberate correction. With plain Bayes ($w = 1$) the three sensors alone moved L by about ± 7 on every request. In our early runs, trust jumped from 0.99 to 0.01 on one malicious request and back to 0.92 on the next clean one, and an honest device fell to 0.016 after two simultaneous false alarms. Several pieces of evidence about the same request are strongly correlated. Multiplying them as if they were

independent overstates the evidence, and tempering with a weight below one is a standard remedy.

3.4.4 Worked Example

Table 3.6 follows one real request through the engine: request `40-DEV-008-1` in the compromised-device scenario with seed 42. The smart meter DEV-008 holds its real secret, so its proof passes, but it sends an `authorized_service` request to the smart camera DEV-002, with which it has no configured link and which lies in another zone. All three sensors raise an alarm.

Table 3.6: Trust calculation for request 40-DEV-008-1 (compromised smart meter, seed 42), taken from the run log.

Step	**Log-odds**
Trust before the request $T = 0.99$, so $L = \ln(T/(1-T))$	$+4.5951$
authorised service (request kind)	$+0.0375$
no configured link DEV-008 $\rightarrow$ DEV-002	-1.6094
cross-zone access (Private $\rightarrow$ Public IoT)	-1.0397
IDS raised an alarm	-0.4703
SIEM raised an alarm	-0.4028
Behaviour Monitor raised an alarm	-0.4336
L after the request	$+0.6767$
Trust $= \sigma(L) \rightarrow$ band Suspicious $\rightarrow$ STEP-UP	0.6630
step-up: fresh ZK proof PASS, MFA FAIL (step_up_fail)	-1.6720
Final L, trust $= \sigma(L) \rightarrow$ Isolated $\rightarrow$ BLOCK, quarantine to t = 70	-0.9953 / 0.2699

The terms add up to -3.9184, so L falls from 4.5951 to 0.6767 and trust falls from 0.99 to 0.6630. That is below the ALLOW threshold of 0.72, so the policy asks for a step-up. The device passes the fresh proof (it has the secret) but fails the multi-factor check. The step-up failure term takes trust to 0.2699, below the BLOCK threshold of 0.45, and the device is quarantined until tick 70. Figures 3.5 and 3.6 show the same request as a pipeline and on the sigmoid curve. Every request in a run can be laid out in this way in the dashboard's *Trust calculation* tab.

The same request appears in the live terminal of the dashboard exactly as in Listing 3.1. The first line shows an earlier, ordinary request by the same device in the same tick: the three silent sensors each add a small positive term, and a small negative `decay` term shows how far the per-tick pull towards the prior had moved L since the device's previous request. The quarantine notice and the dropped requests that follow show the enforcement.

Listing 3.1: Terminal output for DEV-008 around tick 40 (compromised-device scenario, seed 42).

```
[t=040] DEV-008 → CONTROLLER  token_request ×2 (window 20)   ZK:PASS  IDS:-  SIEM:-  BM:-
        Bayes trust 0.99 → 0.99  (log-odds: decay -0.13, IDS· +0.37, SIEM· +0.31, BM· +0.34)  band: Trusted
     → Trusted
        → ALLOW (band Trusted)
[t=040] DEV-008 → DEV-002  authorized_service   ZK:PASS  IDS:alert  SIEM:alert  BM:alert   flags:
     no_topology_edge, cross_zone_access
        Bayes trust 0.99 → 0.66  (log-odds: authorized_service +0.04, no_topology_edge -1.61,
     cross_zone_access -1.04, IDS! -0.47, SIEM! -0.40, BM! -0.43)  band: Trusted → Suspicious
        STEP-UP challenge → ZK:PASS MFA:FAIL  (step_up_fail -1.67)  trust 0.66 → 0.27  band → Isolated  →
     BLOCK
[t=040] DEV-008 QUARANTINED for 30 ticks — links cut
[t=040] DEV-008 → CONTROLLER  read_sensor  DROPPED — DEV-008 is quarantined until t=70
[t=041] DEV-008 → DEV-010  read_sensor  DROPPED — DEV-008 is quarantined until t=70
[t=041] DEV-008 → DEV-004  authorized_service  DROPPED — DEV-008 is quarantined until t=70
```

3.4.5 How Sensor Alarms Are Generated

Because the sensors are simulated, it is important to be precise about how an alarm is produced. For every request and every sensor, the environment draws a uniform random number $u \in [0, 1)$ from the stream keyed by (seed, device, tick, sensor, request index). If the request is malicious, the sensor uses its true-positive rate $p = \text{TP}$. Otherwise it uses its false-positive rate $p = \text{FP}$. The alarm fires if and only if $u < p$. Table 3.7 shows the three draws for the worked request. It was malicious, so each sensor used its TP rate, and all three draws fell below it.

Table 3.7: Sensor draws for request 40-DEV-008-1: an alarm fires when $u < p$.

Sensor	**Rate used p**	**Random draw u**	**$u < p$?**	**Observed**
IDS	0.92 (true-positive rate)	0.1383	yes	alarm
SIEM	0.88 (true-positive rate)	0.2677	yes	alarm
BM	0.90 (true-positive rate)	0.3472	yes	alarm

The controller sees only the result (alarm or silent), never p or u. The draws are stored next to the ground truth so that the dashboard's *Alarm system* view can explain every alarm after the run. They are never passed to either controller.

3.4.6 Decay and Recovery

The per-tick decay of Equation 3.6 has two effects. First, a device whose trust fell and then stays quiet slowly drifts back towards the prior. Without new evidence, the distance $L - \text{logit}(\pi)$ shrinks by a factor λ every tick, so after k ticks

$$L_k - \text{logit}(\pi) = \lambda^k \left(L_0 - \text{logit}(\pi) \right), \tag{3.8}$$

and the evidence loses half of its weight after

$$k_{1/2} = \frac{\ln 0.5}{\ln \lambda} = \frac{\ln 0.5}{\ln 0.98} \approx 34 \text{ ticks.} \tag{3.9}$$

Figure 3.4 shows this for a device left at trust 0.27. It climbs back above the BLOCK threshold after about 20 quiet ticks and above the ALLOW threshold after about 85. Second, decay also pulls a very trusted device *down* towards the prior of 0.8. This is why honest devices do not simply sit at 0.99. Their ordinary requests and silent sensors must keep earning their trust, and a device that stops reporting slowly becomes less trusted.

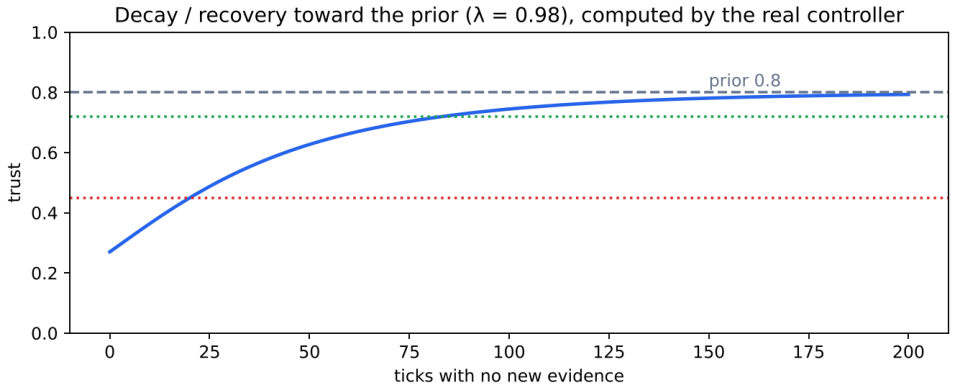


Figure 3.4: Recovery towards the prior with $\lambda = 0.98$ when no new evidence arrives, computed by the real controller.

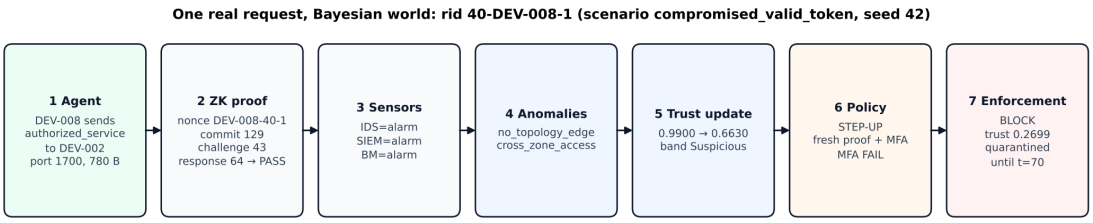


Figure 3.5: The seven stages of request 40-DEV-008-1 in the Bayesian world.

3.5 POLICY ENGINE

The policy engine maps a device's trust to one of three bands and an action. The rules are checked in order and the first match wins (Table 3.8).

A device must cross a threshold by a margin of 0.03 before it changes band (Figure 3.7). This *hysteresis* stops a device that sits near a threshold from flickering between actions. The step-up challenge applies GAZETA's idea that a less trusted requester must

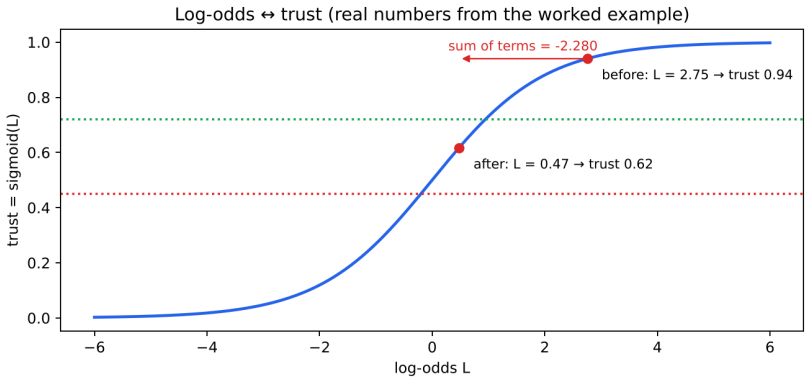


Figure 3.6: Log-odds and trust are linked by the sigmoid. The marked points come from a token-flood request in the project documentation ($L = 2.75 \rightarrow 0.47$, trust $0.94 \rightarrow 0.62$).

Table 3.8: Policy rules (first match wins).

Condition	**Action**
ZK proof fails, or nonce already seen	BLOCK this request (hard gate)
Device on probation	STEP-UP
Trust ≥ 0.72 (Trusted)	ALLOW
$0.45 \leq$ trust < 0.72 (Suspicious)	STEP-UP: fresh proof on a challenge nonce + simulated MFA
Trust < 0.45 (Isolated)	BLOCK and quarantine for 30 ticks

authenticate more strongly [6]. Its outcome is itself evidence: passing adds $+0.93$ to L, and failing adds -1.67. In the environment, an honest device passes the simulated MFA with probability 0.97 and an active attacker with probability 0.15. The controller sees only PASS or FAIL.

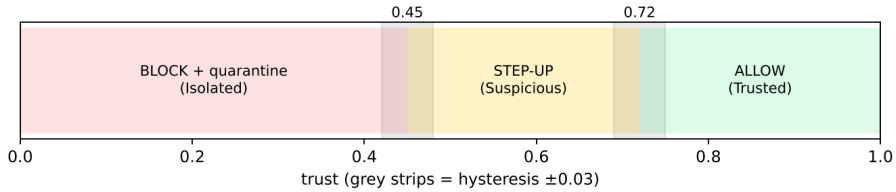


Figure 3.7: Trust bands and actions. The grey strips show the ± 0.03 hysteresis margin.

A quarantined device has its links cut for 30 ticks: every request to or from it is dropped and logged, with no trust update. It is then released on *probation*. Its trust is reset to 0.60, and it must pass a step-up before it can be allowed again. A device that misbehaves during probation goes straight back into quarantine. Figure 3.8 shows the resulting state machine, which is the same in both worlds.

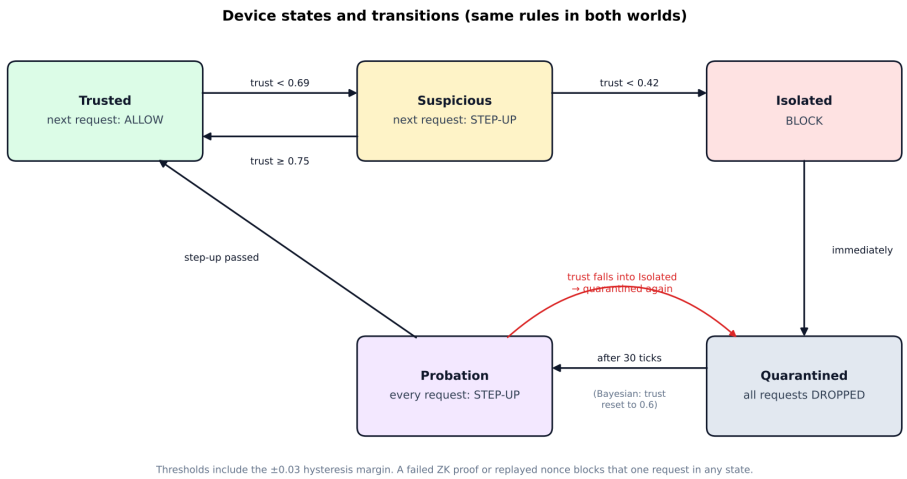


Figure 3.8: Device states and transitions.

3.5.1 Per-Request Processing Algorithm

Algorithm 3.1 summarises how the Bayesian world handles one request, combining the authentication, evidence, trust update and policy steps described above.

Algorithm 3.1 Processing one request in the Bayesian world

Require: observed event e from device d; sensor observations o; ledger of d

if d is quarantined **then**
  drop the request and log it; **return** DROPPED
end if
if VERIFYZK$(y_d, e.\mathit{proof})$ fails **or** $e.\mathit{nonce}$ already seen from d **then**
  add `zk_auth_fail` or `token_replay` evidence; **return** BLOCK ▷ hard gate
end if
remember $e.\mathit{nonce}$; add e to the 20-tick window of d
$E \leftarrow \{\text{request kind}\} \cup$ DERIVEANOMALIES$(e,$ window, baseline, topology, clock$)$
$L \leftarrow \text{logit}(T_d) + \sum_{x \in E} w_e \ln \frac{P(x|\text{legit})}{P(x|\text{mal})} + \sum_s w_s \ln \frac{P(o_s|\text{legit})}{P(o_s|\text{mal})}$
$T_d \leftarrow \min(\max(\sigma(L), 0.01), 0.99)$; band $\leftarrow$ CLASSIFY$(T_d,$ hysteresis $0.03)$
if d on probation **or** band = Suspicious **then**
  run step-up (fresh proof on a challenge nonce + MFA); add `step_up_pass` or `step_up_fail`
  recompute T_d and band
end if
if band = Isolated **then**
  quarantine d for 30 ticks; **return** BLOCK
else if a step-up was run **then**
  return STEP-UP (the request goes through only if the challenge was passed)
else
  return ALLOW
end if

3.6 LLM TRUST ENGINE

The second engine replaces the Bayesian trust calculation with a large language model. It is a complete, independent controller in its own world: it verifies proofs, remembers nonces, keeps its own window and baselines, and enforces its own decisions. Nothing computed by the Bayesian engine ever reaches it. Figure 3.9 shows how a verdict is produced.

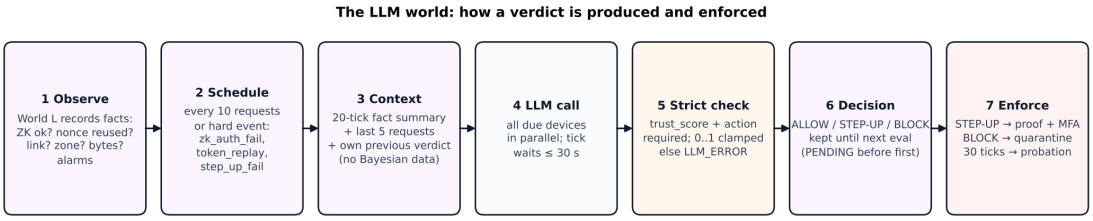


Figure 3.9: How the LLM world produces and enforces a verdict.

3.6.1 Facts-Only Context

When a device is due for evaluation, the LLM world builds a JSON context from its own observations only. The context has the fields device, configured_outgoing_links, current_tick, evaluation_trigger, enforcement_state, previous_llm_assessment, window_ticks, window_facts, recent_requests, sensor_reference, policy_guidance, omitted_fields_mean_zero. It contains the device's public attributes (type, protocol, firmware, zone, sensitivity), its configured links, the LLM's own previous verdict and the device's enforcement state. It also contains a 20-tick fact summary: request count and rate against the device's own baseline, counts per request kind, token requests, distinct destinations and ports, requests without a configured link, zone-changing requests, requests to sensitive targets, maximum payloads against baseline, ZK failures, nonce reuse, night requests, sensor alarm counts and step-up results. Finally it lists the last 5 requests individually. Zero or empty counters are left out, and the context says that omitted fields mean zero. The context never contains a name, a role, an attack label or any Bayesian value, and an automated test walks every context of complete runs to check this. Listing 3.2 shows part of a real context for the compromised smart meter at tick 64 of the multi-attacker scenario. The full context was 2,532 characters of compact JSON.

Listing 3.2: Part of the facts-only context sent for DEV-008 (multi-attacker scenario, seed 0).


```
{
  "device": {"id": "DEV-008", "type": "Meter", "protocol": "LoRaWAN", "firmware": "v3.4.0", "zone": "Private
      IoT", "target_sensitivity": 0.75},
  "configured_outgoing_links": ["DEV-010"],
  "current_tick": 64,
  "evaluation_trigger": "every 10 events",
  "previous_llm_assessment": {"tick": 58, "trust_score": 0.92, "action": "ALLOW", "confidence": 0.6},
  "window_facts": {
    "requests": 30,
    "requests_per_tick": 1.5,
    "baseline_requests_per_tick": 1.2,
    "request_kind_counts": {"read_sensor": 17, "token_request": 1, "normal_telemetry": 9, "
     authorized_service": 3},
    "token_requests": 1,
    "distinct_destinations": 3,
    "distinct_ports": 1,
    "requests_without_configured_link": 3,
    "zone_changing_requests": 3,
    "requests_to_sensitive_targets": 15,
    "max_payload_bytes_by_kind": {"read_sensor": 3277, "token_request": 144, "normal_telemetry": 2644, "
     authorized_service": 909},
    "baseline_mean_payload_bytes_by_kind": {"read_sensor": 298.5, "normal_telemetry": 190.2, "token_request
     ": 140.0},
    "sensor_alarm_counts": {"IDS": 7, "SIEM": 6, "BM": 4}
  },
  "recent_requests": [
    {"tick": 62, "dst": "DEV-004", "kind": "authorized_service", "port": 1700, "bytes": 909, "zk": "PASS", "
     nonce_reused": false, "configured_link": false, "zone_to": "Critical Zone", "target_sensitivity": 0.95,
      "sensor_alarms": ["IDS", "SIEM"]},
    "... (4 more requests)"
  ]
}
```


3.6.2 Prompt and Output Schema

The system prompt tells the model that it is the trust evaluation and policy engine of an IoT zero-trust controller. It explains that it will receive only observed facts about one device, that no label says whether the device is an attacker, that ZK failures and nonce reuse are strong evidence while a valid proof alone is not, and that honest devices occasionally update firmware, reboot and perform maintenance. It must return only a JSON object with five keys: `trust_score` (0 to 1), `action` (ALLOW, STEP-UP or BLOCK), `confidence` (0 to 1), a one-sentence `reason`, and at most four `factors` naming the facts it relied on. The same structure is passed as a JSON schema to constrain decoding: Ollama's `format` field, Gemini's `response_json_schema` and the `response_format` of OpenAI-compatible servers.

3.6.3 Scheduling and Enforcement

Calling an LLM for every request would be far too slow, so a device is evaluated after every 10 of its processed requests, but not again within 3 ticks. It is also evaluated

immediately after a *hard event* (a failed proof, a replayed nonce or a failed step-up), on release from quarantine and when probation ends. All due devices are evaluated in parallel at the end of the tick, and the tick waits for the answers. Between evaluations the LLM world enforces the last verdict: ALLOW and STEP-UP are applied to the device's next requests, and BLOCK quarantines it under the same 30-tick rule as the Bayesian world. Before a device's first evaluation its trust is *PENDING*, and requests with a valid proof are allowed, just as a new device starts in the Trusted band in the Bayesian world. The cryptographic hard gate is identical in both worlds. Algorithm 3.2 summarises one tick of the LLM world.

Algorithm 3.2 One tick of the LLM world

for each request e of device d in this tick **do**
  if d quarantined **then drop** e
  else if proof invalid **or** nonce reused **then** BLOCK e; mark d *due* (hard event)
  else
    record the facts of e in d's window
    enforce d's last LLM verdict (PENDING $\Rightarrow$ ALLOW); if STEP-UP, run the challenge
    if step-up failed **then** mark d *due*
    end if
    count e; **if** d has ≥ 10 new requests **and** last evaluation ≥ 3 ticks ago **then** mark d *due*
  end if
end for
in parallel for each due device d:
  $C \leftarrow$ BuildFactsOnlyContext(d) ▷ no Bayesian value, no labels
  $R \leftarrow$ CallLLM(system prompt, C, JSON schema)
  if R is valid JSON with `trust_score` and a valid `action` **then** store verdict
  else record `LLM_ERROR`, keep previous verdict ▷ never a Bayesian fallback
  if verdict = BLOCK **then** quarantine d for 30 ticks
wait for all calls (up to the timeout), then advance to the next tick

3.6.4 Strict Validation and Failure Handling

Every reply is validated strictly. A missing or non-numeric `trust_score`, an action outside the three allowed values, invalid JSON, a timeout or an unreachable server is recorded as an `LLM_ERROR` of a specific type and counted. The device keeps its previous LLM decision and is retried at its next scheduled point. Out-of-range numbers are clamped into $[0, 1]$ and counted. After repeated connection failures the dashboard shows *LLM offline*, and the LLM world keeps running. The LLM world *never* falls back to a Bayesian value. Doing so would silently turn the comparison into Bayesian versus Bayesian.

3.6.5 Worked Evaluation

Listing 3.2 showed the context for the compromised smart meter at tick 64 of the multi-attacker scenario, which was produced by the trigger "every 10 events". Three of its 30 recent requests had no configured link and changed zone, half of them went to sensitive targets, its largest payloads were more than ten times its baseline, and the sensors had raised 17 alarms. Its previous verdict was ALLOW. Listing 3.3 shows the reply returned by the rule-based stand-in for this context.

Listing 3.3: Reply to the context of Listing 3.2 (rule-based stand-in).

```
{
 "trust_score": 0.6200000000000001,
 "action": "STEP-UP",
 "risk_score": 0.3799999999999999,
 "confidence": 0.8,
 "reason": "3 requests without configured link",
 "factors": [
  "3 requests without configured link"
 ],
 "behavior_assessment": []
}
```

The reply passed strict validation: the trust score is a number in $[0, 1]$ and the action is one of the three allowed values. The LLM world therefore stored trust 0.62 and action STEP-UP, and enforced it from the next tick. At tick 65 every request of the meter was challenged. The meter passed the fresh proof but failed the multi-factor check, so each of those requests was blocked. A failed step-up is a hard event, so the device was evaluated again at the end of the same tick, and the stand-in, seeing the same single rule fire, again chose STEP-UP rather than quarantine. The example also shows the weakness of the stand-in discussed in Chapter 4. Only one of its six rules fired (requests without a configured link), so it asked for a step-up instead of blocking, even though the window also contained zone changes, oversized payloads and many sensor alarms. A real language model receives exactly the same facts and could weigh them differently.

3.6.6 Model Providers

The engine supports three providers through one interface:

- **Ollama (local)**, the default, with `llama3.2:3b` [26]. Requests use temperature 0, a 160-token output cap, a 4096-token context window, `keep_alive` of 30 minutes so that the model stays loaded, and thinking mode switched off for models that have one.

- **Google Gemini (API)** [27], through the official `google-genai` SDK. The model ID is configurable, with a Flash-Lite model as the default. Requests send the JSON schema, a 160-token cap and a low thinking level. A client-side limiter keeps calls under a configurable rate (10 per minute by default). Rate-limit errors are retried with exponential backoff (1, 2 and 4 seconds plus jitter), and a daily-quota error takes the LLM world offline. The API key is read from an environment variable or a password field. It is masked whenever it is shown and never written to logs or exports.
- **OpenAI-compatible API**, which also covers local servers such as llama.cpp.

For the Gemini provider, every call goes through the client-side rate limiter and the retry rules summarised in Algorithm 3.3. A per-minute rate limit only delays a call, while a daily quota error takes the LLM world offline for the rest of the run. In both cases the simulation keeps running and the Bayesian world is unaffected.

Algorithm 3.3 One Gemini call with rate limiting and back-off

Require: context C; limit of R requests per minute; retries $\leftarrow 0$; thinking $\leftarrow$ low
if daily quota already exhausted **then raise** quota_exhausted
end if
loop
    wait until fewer than R calls were made in the last 60 seconds; record this call
    send GenerateContent(model, C, system prompt, JSON MIME type, schema, 160 tokens, thinking)
    if success **then return** reply text and token usage
    end if
    if HTTP 429 **and** message mentions a per-day quota **then**
        mark quota exhausted; **raise** quota_exhausted ▷ LLM world goes offline
    else if HTTP 429 **and** retries < 3 **then**
        sleep 2^{retries} s + random jitter in $[0, 0.5]$ s; retries $\leftarrow$ retries $+1$
    else if HTTP 429 **then raise** rate_limited
    else if HTTP 400 **and** thinking is set **then** thinking $\leftarrow$ none; log a warning ▷ retry once
    else raise unavailable (key removed from the message)
    end if
end loop

3.6.7 Rule-Based Stand-in for Pipeline Validation

Running a real LLM over hundreds of seeded runs needs a GPU or a large API quota. To check the whole comparison pipeline without either, the project includes a *rule-*

based stand-in. It reads exactly the same facts-only context and answers in exactly the same JSON format, but it decides with six fixed rules instead of a language model. A rule fires on more than one ZK failure, any nonce reuse, more than two requests without a configured link, more than six token requests, more than five distinct ports, or a request rate above three times the baseline. Each rule that fires lowers trust from 0.92 by 0.3, and the usual thresholds map trust to an action. The stand-in is deterministic and makes no errors, which makes it ideal for testing, but **it is not an LLM, and its numbers say nothing about how a real LLM would perform.** Wherever this report shows LLM-world results produced with the stand-in, they are labelled as such.

3.7 TWIN-WORLD ISOLATION AND REPRODUCIBILITY

The comparison is only fair if both engines see the same traffic and neither can learn from the other. ZT-IoT Sentinel therefore runs two complete *worlds* side by side (Figure 3.10). World B uses the Bayesian controller and World L the LLM controller. Each world deep-copies the scenario and has its own devices, agents, ledger, nonce memory, windows, baselines, quarantines and log. The worlds advance in lockstep, one tick at a time.

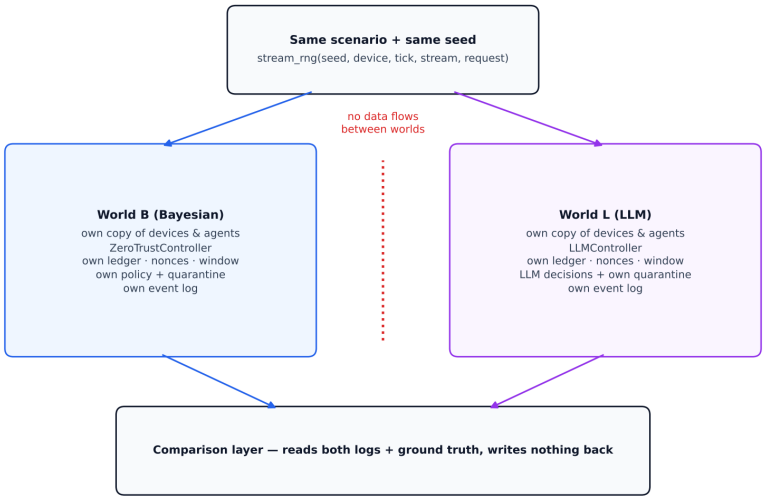


Figure 3.10: Twin worlds: identical inputs, no shared state, and a comparison layer that only reads.

Identical inputs come from how randomness is generated. There is no global random number generator whose consumption order could drift between the worlds. Every

random draw (agent behaviour, sensor alarms, proof randomness, challenge proofs and MFA outcomes) comes from `stream_rng(seed, device, tick, stream, index)`, a generator seeded from exactly those five values. Both worlds therefore generate identical traffic. The only difference in what they *process* comes from their own enforcement: requests from a device that one world has quarantined are dropped in that world only. Every request carries an identifier such as `40-DEV-008-1`, which is the same in both worlds, so the comparison layer can line up the two decisions on each request. That layer reads both logs and the ground truth, and writes nothing back.

3.8 EVALUATION METRICS

The same metric definitions are applied to both worlds, each from its own log. Ground truth is used only after the run. The main metrics are listed in Table 3.9.

Table 3.9: Evaluation metrics.

Metric	**Definition**
Detected	The attacker was quarantined at or after its attack start.
Ticks to BLOCK	Ticks from the attack start to the first quarantine.
Honest quarantined (FPR)	Share of honest and noisy-honest devices that were ever quarantined.
Honest stepped-up	Share of honest and noisy-honest devices that were ever challenged.
Hard-gate blocks	Single requests blocked because the proof failed (counted separately).
Precision, recall, F1	At device level; positive = quarantined at least once.
Per-request accuracy	Share of requests whose policy action matched the truth (blocked or challenged if and only if the sender was an active attacker).
Brier score	$\frac{1}{N}\sum_i (T_i - y_i)^2$ over device-ticks, with $y_i = 1$ for legitimate and 0 for active attackers [31].
Separation gap	Mean trust of honest devices minus mean trust of active attackers.
Agreement	Share of request ids on which both worlds took the same policy action.
Consistency	Share of requests with an identical decision when the same seed is run three times.
Latency	Bayesian: time per request; LLM: wall time per evaluation.
Factor support	Share of the LLM's stated factors that match facts present in its window.

3.9 IMPLEMENTATION

3.9.1 Technology Stack

The system is written in Python. Table 3.10 lists the main components.

Table 3.10: Technology stack.

Component	Use
Python 3	Simulator, controllers, metrics, experiments
Streamlit	Interactive dashboard (Live and Classic modes)
Plotly, Matplotlib	Live charts; report and batch figures
pandas, NumPy, NetworkX	Tables, statistics, topology
Ollama	Local LLM serving (`llama3.2:3b` by default)
google-genai SDK	Optional Google Gemini provider
pytest	158 automated tests

The code is organised into small modules: `agents.py` (behaviour models), `live_sim.py` (environment and clock), `zk_auth.py` (proofs), `controller.py` (Bayesian engine and policy), `live_llm.py` and `gemini_engine.py` (LLM engine and providers), `twin.py` (twin worlds), `comparison.py` and `metrics.py` (scoring), `behaviour_analysis.py` (attack-kind signatures and the calculation view), and the dashboard in `app.py`, `live_ui.py` and `calc_ui.py`. Headless experiments are run with `experiments/run_batch.py`. Table 3.11 lists all simulation and engine parameters.

3.9.2 Dashboard

The Streamlit dashboard is how we, and an examiner, can watch the system work. It has two modes.

3.9.2.1 Live Mode

In Live mode the user chooses a scenario, a seed and the engines to run (Bayesian only, LLM only, or both), then starts, pauses or steps the simulation one tick at a time. Thresholds, the LLM provider and its settings can be changed from the sidebar. The main area has four tabs.

Live. A terminal-style event log shows every decision as one short line. Routine allowed traffic is folded into a single "n requests allowed" line per tick, and the full list of log-odds terms appears on demand. Next to it are a device status table, the network

Table 3.11: Simulation and engine parameters (`core/live_config.py`).

Parameter	**Value**
Prior trust of every enrolled device	0.8
Trust floor / ceiling	0.01 / 0.99
Decay factor λ (per tick)	0.98
Event weight w_e	0.5
Sensor weight w_s	0.15
ALLOW threshold (Trusted)	0.72
BLOCK threshold (Isolated)	0.45
Hysteresis margin	0.03
Quarantine length (ticks)	30
Trust on release (probation)	0.6
Sliding window / baseline warm-up (ticks)	20 / 20
Free token requests per window	3
Rate spike: factor / minimum excess	$3\times$ / +20
Scan: distinct destinations / ports	> 4 / > 5
Unusual payload factor	$4\times$ baseline
Sensitive target	sensitivity ≥ 0.75
MFA pass rate: honest / attacker	0.97 / 0.15
Ticks per simulated day / night starts at	100 / 80
LLM: evaluate every N requests	10
LLM: minimum ticks between routine evaluations	3
LLM: recent requests listed in the context	5
LLM: maximum output tokens	160
LLM: temperature / timeout (s)	0.0 / 30
LLM: parallel calls per tick	4
Gemini: client-side requests per minute	10

topology coloured by device state, and trust curves. Figure 3.11 shows the Bayesian world alone, and Figure 3.12 shows both worlds side by side.

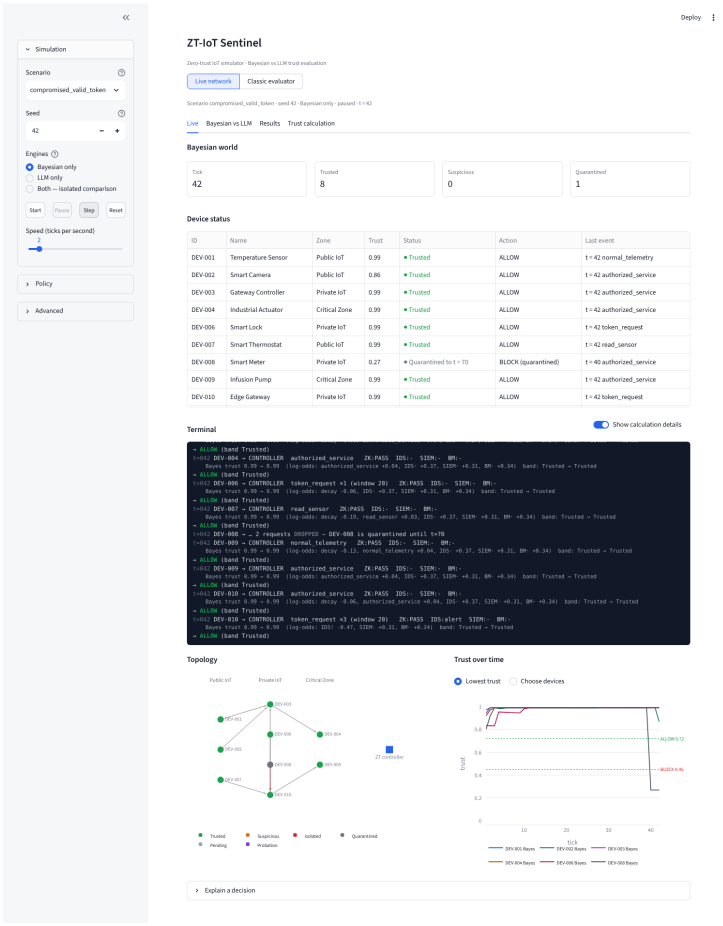


Figure 3.11: Live tab, Bayesian world only (compromised-device scenario after 42 ticks).

Bayesian vs LLM. After a run, this tab computes every comparison metric of Table 3.9 for both worlds, with the difference and which engine did better (Figure 3.13). The *Repeat run* button measures consistency, and *Prepare comparison export* writes CSV files, a Markdown report and the charts into one archive.

Results. This tab scores one world against ground truth: attackers detected, false-negative rate, honest devices quarantined or stepped up, and F1. It also offers downloads of the terminal log, the trust history, the metrics and a summary (Figure 3.14).

Trust calculation. This tab explains any single request in full. For the Bayesian world it lists every log-odds term together with the observed fact behind it, re-adds the terms and checks the result against the log (Figure 3.15). For the LLM world it shows the exact context sent, the raw reply and whether each stated factor is supported by the window. The *Attacks by behaviour* sub-tab infers each attacker's kind from observed facts

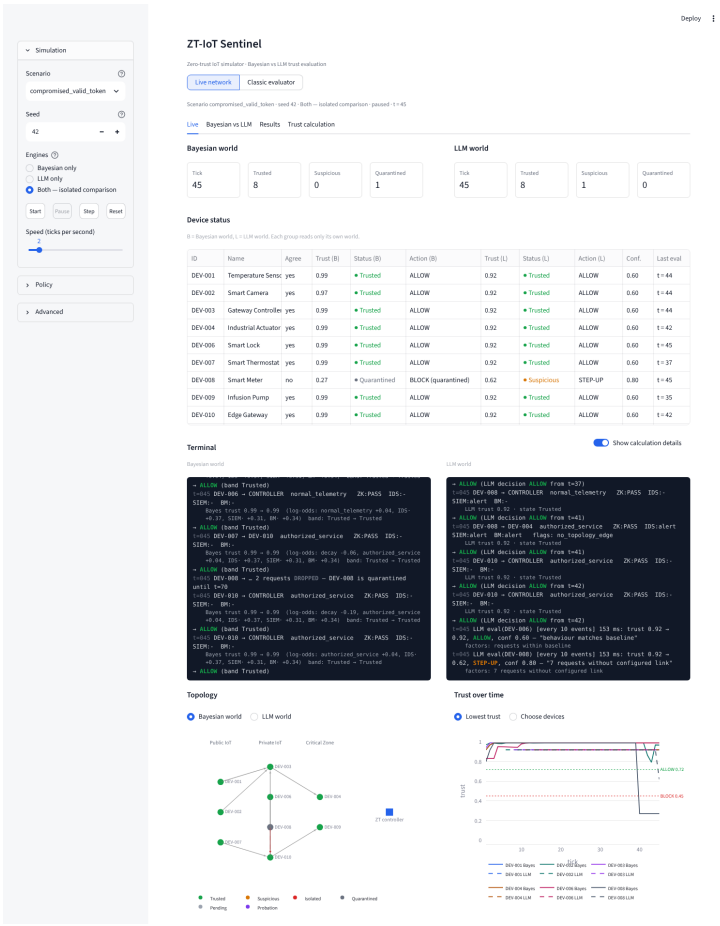


Figure 3.12: Live tab with both worlds running side by side.

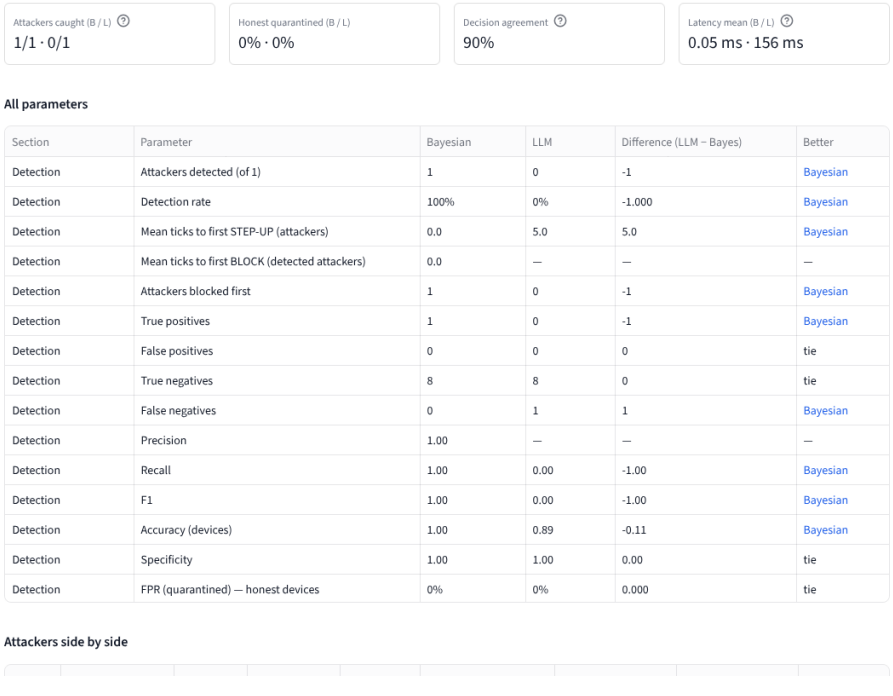


Figure 3.13: Bayesian vs LLM tab: headline metrics and the attackers side by side.

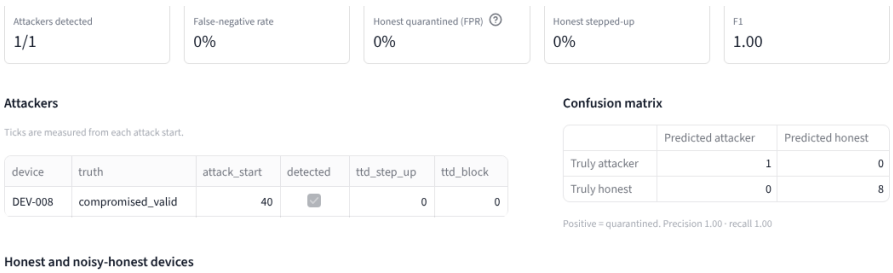


Figure 3.14: Results tab: detection metrics computed from ground truth after the run.

only and compares how the two engines reacted (Figure 3.16). The *Alarm system* sub-tab shows the sensor draws behind every alarm and the observed true- and false-positive rates for the whole run (Figure 3.17).

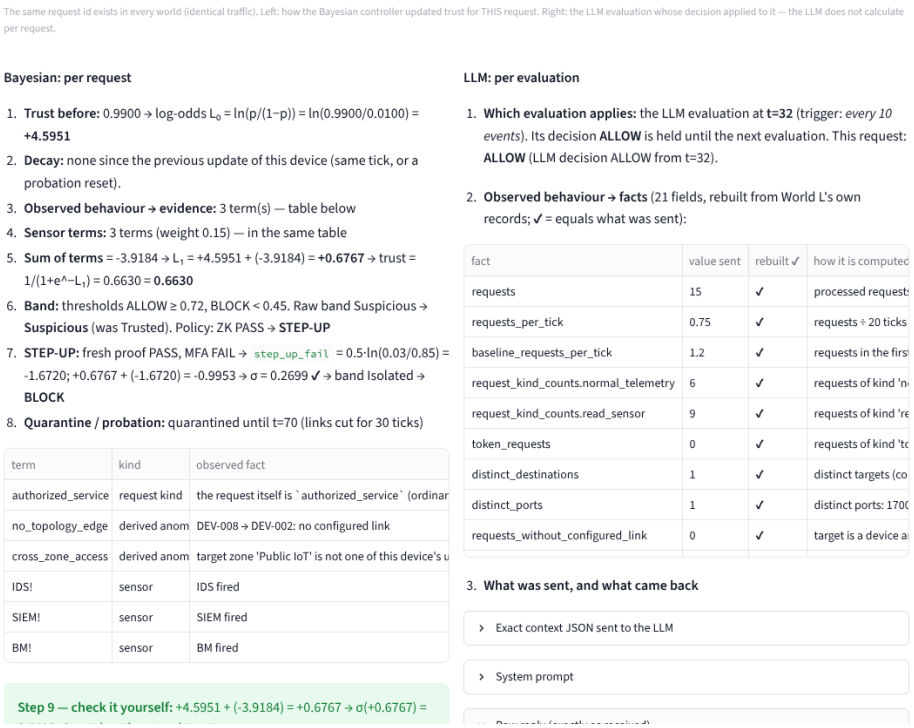
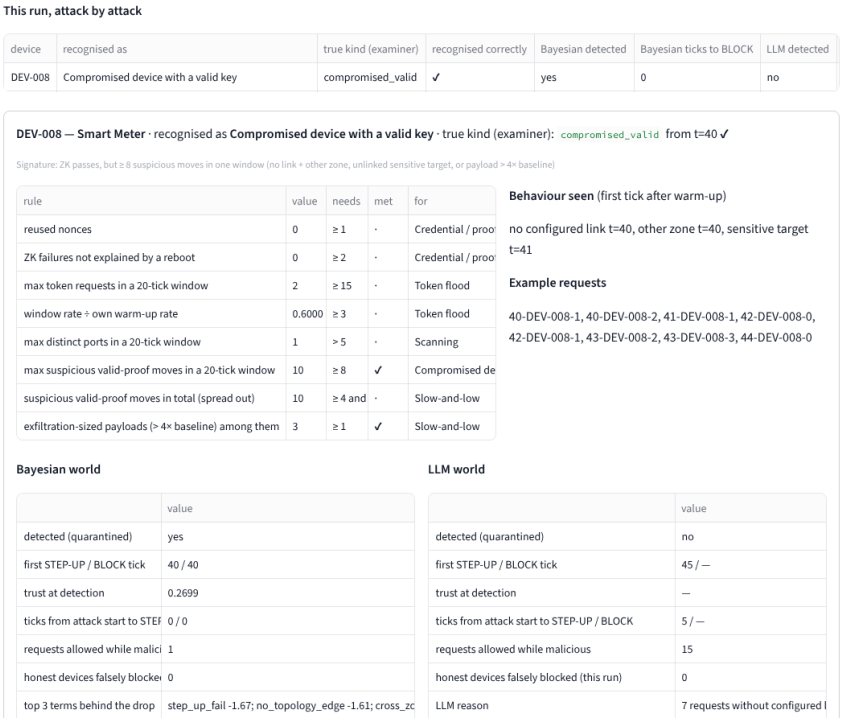


Figure 3.15: Trust calculation tab: the same request id shown in each world.

3.9.2.2 Classic Mode

Classic mode is the project's original single-request evaluator. The user picks a source device, a target and whether to simulate an invalid proof. One request is then authenticated with a zero-knowledge proof and judged by the Bayesian engine, the LLM, or both, with the result, the reasoning and the decision path shown side by side (Figure 3.18). A device catalogue lets the user add new devices to the network. Classic mode is useful for explaining the idea of the project in a few clicks, but its suspicious-behaviour input



This run, attack by attack

device	recognised as	true kind (examiner)	recognised correctly	Bayesian detected	Bayesian ticks to BLOCK	LLM detected
DEV-008	Compromised device with a valid key	compromised_valid	✓	yes	0	no

DEV-008 — Smart Meter · recognised as **Compromised device with a valid key** · true kind (examiner): `compromised_valid` from t=40 ✓

Signature: ZK passes, but ≥ 8 suspicious moves in one window (no link + other zone, unlinked sensitive target, or payload > 4× baseline)

rule	value	needs	met	for
reused nonces	0	≥ 1	·	Credential / proo
ZK failures not explained by a reboot	0	≥ 2	·	Credential / proo
max token requests in a 20-tick window	2	≥ 15	·	Token flood
window rate ÷ own warm-up rate	0.6000	≥ 3	·	Token flood
max distinct ports in a 20-tick window	1	> 5	·	Scanning
max suspicious valid-proof moves in a 20-tick window	10	≥ 8	✓	Compromised de
suspicious valid-proof moves in total (spread out)	10	≥ 4 and	·	Slow-and-low
exfiltration-sized payloads (> 4× baseline) among them	3	≥ 1	✓	Slow-and-low

Behaviour seen (first tick after warm-up)

no configured link t=40, other zone t=40, sensitive target t=41

Example requests

40-DEV-008-1, 40-DEV-008-2, 41-DEV-008-1, 42-DEV-008-0, 42-DEV-008-1, 43-DEV-008-2, 43-DEV-008-3, 44-DEV-008-0

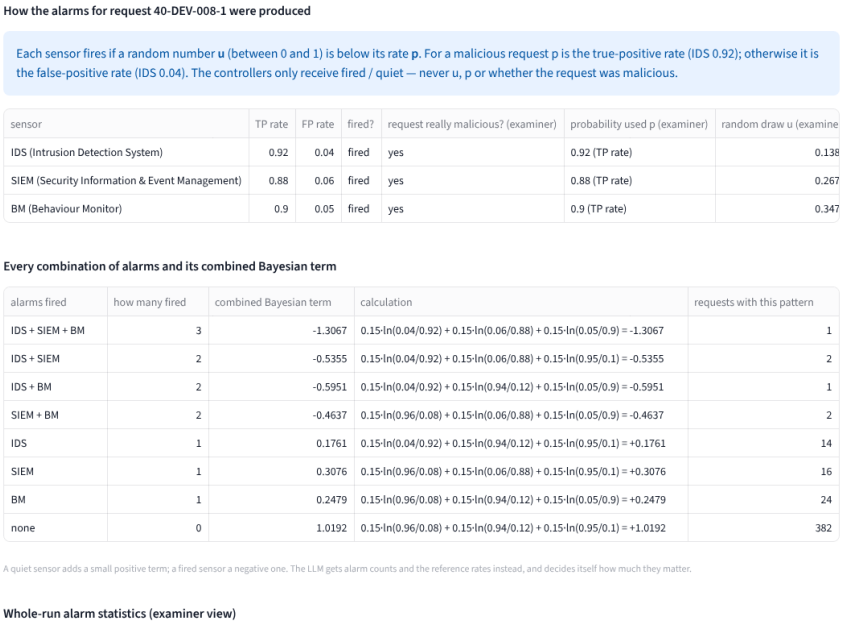
Bayesian world

	value
detected (quarantined)	yes
first STEP-UP / BLOCK tick	40 / 40
trust at detection	0.2699
ticks from attack start to STEP	0 / 0
requests allowed while malici	1
honest devices falsely blocke	0
top 3 terms behind the drop	step_up_fail -1.67; no_topology_edge -1.61; cross_zc

LLM world

	value
detected (quarantined)	no
first STEP-UP / BLOCK tick	45 / —
trust at detection	—
ticks from attack start to STEP-UP / BLOCK	5 / —
requests allowed while malicious	15
honest devices falsely blocked (this run)	0
LLM reason	7 requests without configured l

Figure 3.16: Attacks by behaviour: inferred attack kind and the evidence each engine reacted to.



How the alarms for request 40-DEV-008-1 were produced

Each sensor fires if a random number **u** (between 0 and 1) is below its rate **p**. For a malicious request p is the true-positive rate (IDS 0.92); otherwise it is the false-positive rate (IDS 0.04). The controllers only receive fired / quiet — never u, p or whether the request was malicious.

sensor	TP rate	FP rate	fired?	request really malicious? (examiner)	probability used p (examiner)	random draw u (examine
IDS (Intrusion Detection System)	0.92	0.04	fired	yes	0.92 (TP rate)	0.138
SIEM (Security Information & Event Management)	0.88	0.06	fired	yes	0.88 (TP rate)	0.267
BM (Behaviour Monitor)	0.9	0.05	fired	yes	0.9 (TP rate)	0.347

Every combination of alarms and its combined Bayesian term

alarms fired	how many fired	combined Bayesian term	calculation	requests with this pattern
IDS + SIEM + BM	3	-1.3067	0.15·ln(0.04/0.92) + 0.15·ln(0.06/0.88) + 0.15·ln(0.05/0.9) = -1.3067	1
IDS + SIEM	2	-0.5355	0.15·ln(0.04/0.92) + 0.15·ln(0.06/0.88) + 0.15·ln(0.95/0.1) = -0.5355	2
IDS + BM	2	-0.5951	0.15·ln(0.04/0.92) + 0.15·ln(0.94/0.12) + 0.15·ln(0.05/0.9) = -0.5951	1
SIEM + BM	2	-0.4637	0.15·ln(0.96/0.08) + 0.15·ln(0.06/0.88) + 0.15·ln(0.05/0.9) = -0.4637	2
IDS	1	0.1761	0.15·ln(0.04/0.92) + 0.15·ln(0.94/0.12) + 0.15·ln(0.95/0.1) = +0.1761	14
SIEM	1	0.3076	0.15·ln(0.96/0.08) + 0.15·ln(0.06/0.88) + 0.15·ln(0.95/0.1) = +0.3076	16
BM	1	0.2479	0.15·ln(0.96/0.08) + 0.15·ln(0.94/0.12) + 0.15·ln(0.05/0.9) = +0.2479	24
none	0	1.0192	0.15·ln(0.96/0.08) + 0.15·ln(0.94/0.12) + 0.15·ln(0.95/0.1) = +1.0192	382

A quiet sensor adds a small positive term; a fired sensor a negative one. The LLM gets alarm counts and the reference rates instead, and decides itself how much they matter.

Whole-run alarm statistics (examiner view)

Figure 3.17: Alarm system view: how the alarms for one request were drawn, and whole-run alarm statistics.

comes from a device's configured role. It is therefore a demonstration only, and all results in this report come from Live mode, where the controllers never see roles.

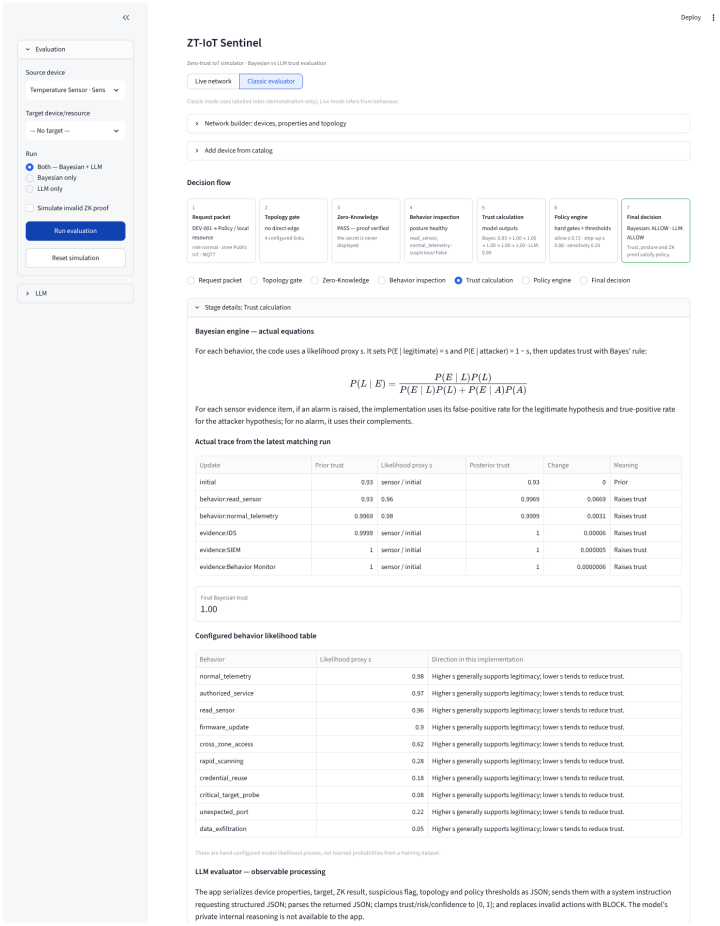


Figure 3.18: Classic mode: one request evaluated by both engines.

3.9.3 Testing

The project has 158 automated tests. Among other things, they check that:

- the controller cannot see ground truth;
- the two worlds share no objects, and changing one does not affect the other;
- every LLM context of complete runs is free of forbidden keys and values;
- every record, terminal line and trust history matches stored fingerprints, so later changes cannot silently alter a decision;
- every term of every logged Bayesian update re-adds to the recorded trust within 10^{-9};

- the Gemini provider sends the expected request, retries correctly, respects its rate limit and never leaks its key. These tests use a mocked client, so they need no network.

Chapter 4

RESULTS

This chapter presents what ZT-IoT Sentinel achieved in our experiments. We first describe the experimental setup. We then report the detection performance of the Bayesian engine over all scenarios, look at trust trajectories, check the sensor model, and test our own assumptions with an ablation study. Finally we compare the Bayesian world with the LLM world, scenario by scenario and attack by attack, and discuss the results. Every table and figure in this chapter is generated directly from the experiment outputs by a script in the report folder (`tools/make_assets.py`), so no number was typed in by hand.

4.1 EXPERIMENTAL SETUP

All experiments were run on a 2-core Linux machine without a GPU, using the project's batch runner `experiments/run_batch.py`. Three sets of runs were made:

1. **Bayesian engine:** all seven scenarios, 30 seeds each (210 runs, 240 attacker runs), with the default configuration.
2. **Ablation:** the same 210 runs with the sensor weight set to 0, so that IDS, SIEM and behaviour-monitor alarms are ignored and the engine relies on behaviour alone.
3. **Twin-world comparison:** all seven scenarios, 10 seeds each, with both worlds running, and the first seed of each scenario repeated three times to measure consistency. The LLM world in these runs used the rule-based LLM stand-in described in Section 3.6.7.

Each run lasts the scenario's suggested length (160 ticks, 200 for multi-attacker and 250 for slow-and-low). Because every random draw is keyed by the seed, each run can be repeated exactly. The commands are:

```
python experiments/run_batch.py --scenario all --seeds 30
python experiments/run_batch.py --scenario all --seeds 30 --sensor-weight 0
python experiments/run_batch.py --mode both --fake-llm --scenario all --seeds 10 --repeat 3
```

About the LLM-world numbers. We could not run a real language model over hundreds of seeded runs on the available hardware. The LLM-world results in Sections 4.7 and 4.9 were therefore produced with the rule-based stand-in, which uses the same facts-only context, the same JSON protocol and the same enforcement as a real LLM. They show that the comparison pipeline works end to end and how to read its output. **They are not measurements of a real LLM.** Running the last command above with `--provider ollama` or `--provider gemini` instead of `--fake-llm`, and rebuilding this report, replaces them with real-model results.

4.2 DETECTION PERFORMANCE OF THE BAYESIAN ENGINE

Table 4.1 summarises the Bayesian engine over 30 seeds per scenario. Across all 240 attacker runs, the attacker was quarantined in every case. For four of the attack types, the median time from the attack start to quarantine was zero ticks: the attacker was isolated on the same tick it started misbehaving. Meanwhile only 0.6–1.7% of honest and noisy-honest devices were ever quarantined, and the device-level F1 score ranged from 0.956 to 0.995.

Table 4.1: Bayesian engine results over 30 seeds per scenario.

Scenario	Attacker runs	Detection rate (%)	Ticks to BLOCK median (max)	Honest quarantined (%)	Honest stepped-up (%)	Mean F1
Normal day	0	—	—	1.5	3.3	—
Compromised (valid token)	30	100	0 (1)	0.8	2.9	0.978
Credential replay	30	100	0 (1)	1.3	3.3	0.967
Scanning	30	100	0 (0)	1.3	2.5	0.967
Token flood	30	100	0 (0)	1.3	2.5	0.967
Slow-and-low	30	100	8.5 (64)	1.7	5.8	0.956
Multi-attacker	90	100	0 (1)	0.6	1.7	0.995

Figure 4.1 shows the full distribution of detection times. The blatant attacks (scanning, token flood, replay and the compromised device) are caught at once, because a single malicious tick produces several strong pieces of evidence: a missing link, a zone change, a scan pattern or a replayed nonce, together with the sensor alarms. The slow-and-low attacker is the exception. It makes one malicious move on only about 15% of ticks and hides it among honest traffic, so the decay of Equation 3.6 and the positive terms from

clean requests partly undo each piece of evidence before the next one arrives. It was still caught in every run, but after a median of 8.5 ticks and up to 64 ticks in the worst run.

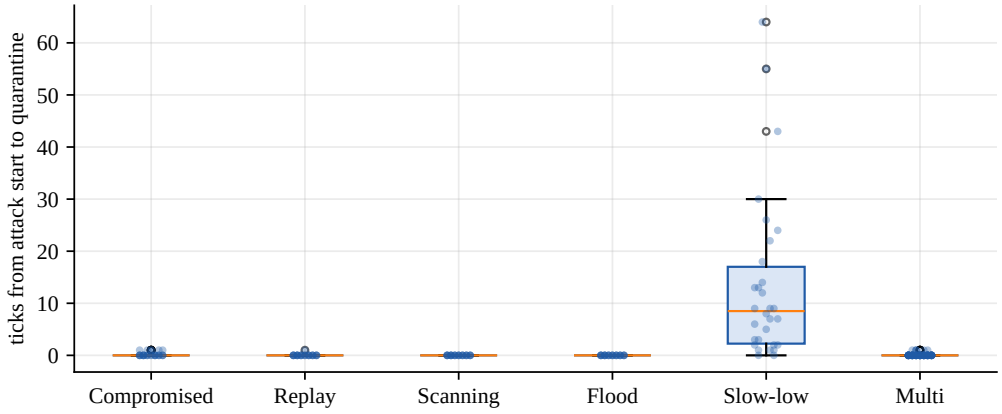


Figure 4.1: Ticks from attack start to quarantine for every attacker run (Bayesian engine, 30 seeds per scenario).

The few honest quarantines are almost all noisy-honest devices. A maintenance access to an unlinked device in another zone, or a reboot followed by failed proofs and then a failed MFA, can push such a device below the BLOCK threshold. In the normal-day scenario, honest devices also suffered on average 9.43 *hard-gate* blocks per run. These are single requests rejected because a proof failed after a reboot. They are deliberately counted separately, since they block one request rather than isolating the device.

The averages in Table 4.1 hide how much individual runs differ. Figure 4.2 shows the distribution over the 30 seeds of each scenario. In most runs no honest device was quarantined at all. A run that does quarantine one loses exactly one device out of its six to nine honest devices, which appears as a single point between 11% and 17%. The F1 score behaves the same way. It is exactly 1.0 in 164 of the 180 runs that contain an attacker, and it drops only when such a false quarantine happens, never because an attacker was missed.

4.3 TRUST TRAJECTORIES

Figure 4.3 shows how trust evolves in the multi-attacker scenario (seed 0), where a camera starts scanning at $t = 35$, the smart meter is compromised at $t = 60$ and the thermostat starts flooding at $t = 90$. In the Bayesian world (top), trust of honest devices stays close to the ceiling, with only small dips. Each attacker's trust collapses on the tick its attack starts, and the device is quarantined. When a quarantine ends, the device is

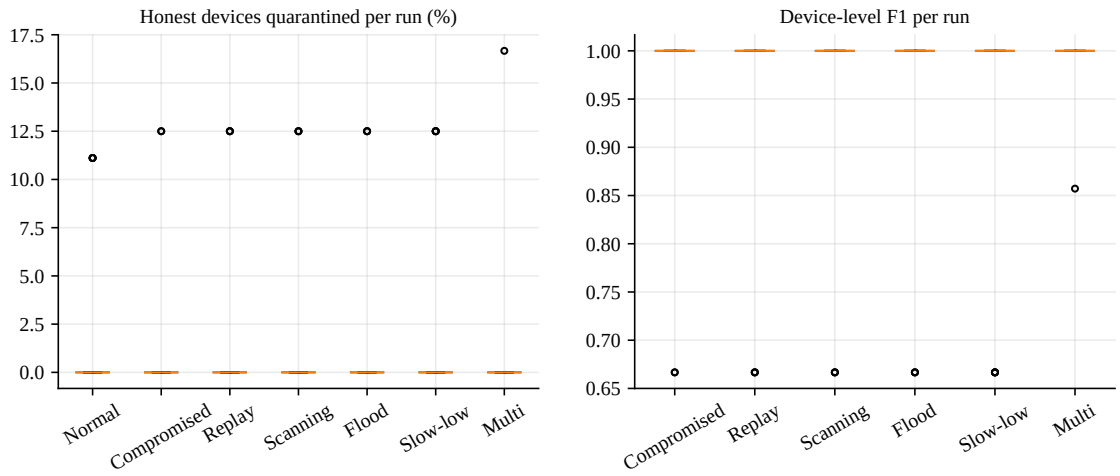


Figure 4.2: Variation across seeds: share of honest devices quarantined and device-level F1 for every run of every scenario (Bayesian engine).

released on probation with trust 0.60. Because these attackers keep attacking, they fail the probation step-up within the same tick and are quarantined again. This is why their curves stay in the red band, with a step every 30 ticks. The noisy-honest edge gateway (green) stays in the Trusted band for the whole run, despite its occasional firmware updates and maintenance accesses.

The bottom panel shows the same run in the LLM world. Its trust is a step function, because it changes only when the device is evaluated, and it is missing (PENDING) until the first evaluation. This is a structural property of any LLM engine that is called periodically rather than on every request, whatever model is used.

4.4 SENSOR EVIDENCE: CONFIGURED VERSUS OBSERVED

Because the sensors are simulated, we checked that they behave as configured. Table 4.2 compares the configured rates with the rates actually observed over 458,924 requests in the Bayesian runs, and Figure 4.4 shows their spread across runs. The observed true- and false-positive rates match the configuration closely. The table also shows something easy to overlook. Because malicious requests are rare compared with legitimate ones, even these good sensors have an alarm precision of only about 0.6–0.65: roughly one alarm in three is false. This is exactly why the engine treats an alarm as weighted evidence and never as a verdict.

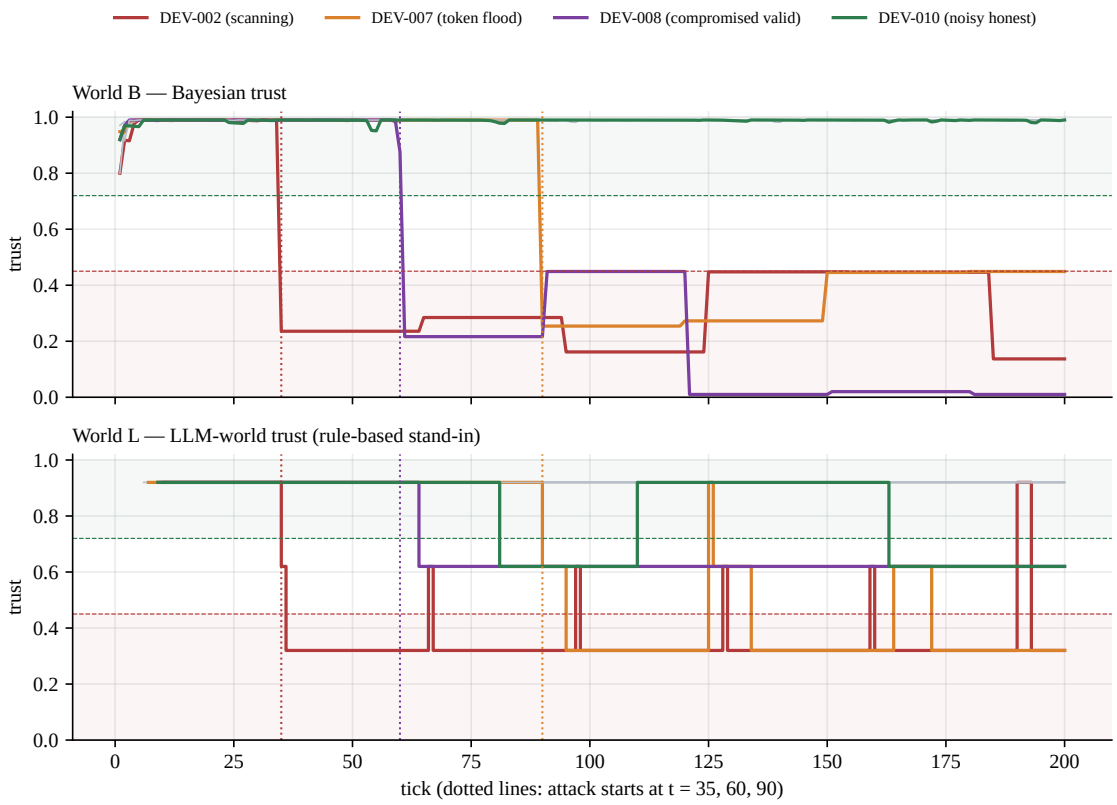


Figure 4.3: Trust over time in the multi-attacker scenario (seed 0). Top: Bayesian world. Bottom: LLM world driven by the rule-based stand-in (step lines, because trust changes only at evaluations; gaps are PENDING). Shaded bands: Trusted (green) and Isolated (red).

Table 4.2: Configured and observed sensor rates (Bayesian runs, all scenarios and seeds).

Sensor	**TP rate (configured)**	**Observed TP (mean ± sd)**	**FP rate (configured)**	**Observed FP (mean ± sd)**	**Alarm precision**	**Requests observed**
IDS	0.92	0.919 ± 0.028	0.04	0.040 ± 0.005	0.647	458,924
SIEM	0.88	0.882 ± 0.026	0.06	0.059 ± 0.006	0.598	458,924
Behaviour Monitor (BM)	0.9	0.897 ± 0.026	0.05	0.050 ± 0.005	0.620	458,924

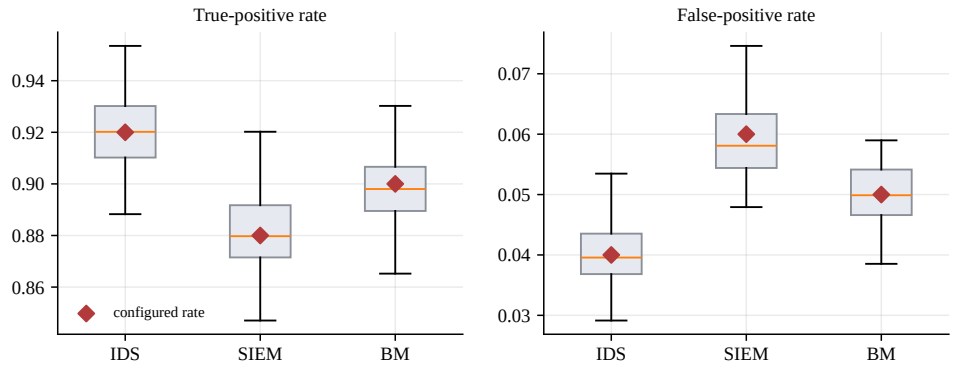


Figure 4.4: Distribution of observed sensor rates per run; diamonds mark the configured rates.

4.5 ABLATION: BEHAVIOUR-ONLY EVIDENCE

The sensors in our simulator are driven by ground truth and are conditionally independent, which is optimistic. To see how much of our result depends on them, we repeated all 210 runs with the sensor weight set to zero (Table 4.3 and Figure 4.5).

Table 4.3: Ablation: Bayesian engine with and without sensor evidence (30 seeds per scenario).

	Honest quarantined (%)		**Mean F1**		**Max ticks to BLOCK**	
Scenario	**sensors**	**no sensors**	**sensors**	**no sensors**	**sensors**	**no sensors**
Normal day	1.5	43.0	—	—	—	—
Compromised (valid token)	0.8	36.2	0.978	0.414	1	1
Credential replay	1.3	46.7	0.967	0.351	1	21
Scanning	1.3	35.8	0.967	0.414	0	18
Token flood	1.3	35.8	0.967	0.417	0	1
Slow-and-low	1.7	50.0	0.956	0.334	64	28
Multi-attacker	0.6	16.1	0.995	0.864	1	23

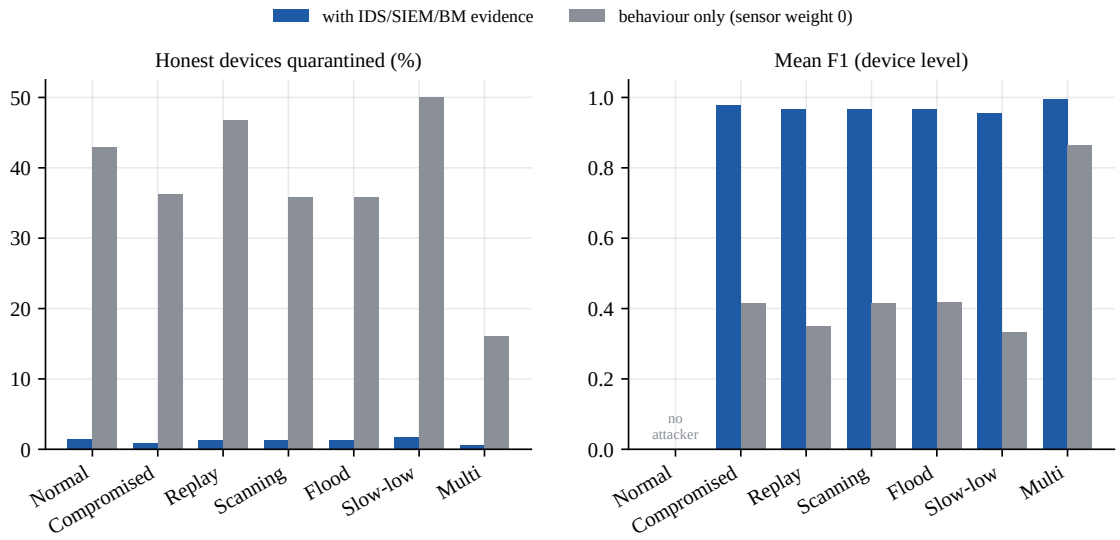


Figure 4.5: Effect of removing sensor evidence on honest quarantines and on F1.

The result is clear and, for us, the most important lesson of the project. Without sensors, every attacker is still caught, from behaviour alone, and the slow-and-low attacker is even caught sooner (maximum 28 ticks instead of 64). But honest quarantines rise from 0.6–1.7% to 16–50% (43% on a normal day), and F1 falls to 0.334–0.864. With sensors present, every clean request adds about $+1.0$ to the log-odds (three "no alarm" terms). That cushion absorbs the legitimate but odd events of noisy-honest devices. Without it, a single maintenance access (no link plus cross-zone, about -2.7 to -3.8 in log-odds) or a

reboot's failed proofs can push a device straight below the BLOCK threshold. In short, the low false-positive rate in Table 4.1 is partly a consequence of our optimistic sensor model, and it should be read that way.

4.6 SENSITIVITY ANALYSIS

The ablation removed one source of evidence completely. To see how the engine depends on its main parameters more generally, we varied four of them one at a time, keeping all others at their defaults. These were the decay factor λ, the event weight w_e, the sensor weight w_s and the BLOCK threshold. For each of the 19 settings we ran all seven scenarios with 10 seeds (70 runs per setting). Table 4.4 gives the results and Figure 4.6 plots the two most important measures, honest quarantines and mean F1. Every setting detected every attacker. The differences lie in how fast the attackers were caught and in how many honest devices were caught by mistake.

Table 4.4: One-at-a-time sensitivity analysis of the Bayesian engine (10 seeds per scenario per setting).

Parameter	Value	Detection (%)	Ticks to BLOCK mean (max)	Slow-and-low median ticks	Honest quarantined (%)	Mean F1	Mean trust honest	Mean trust attacker
Decay factor λ	0.9	100	1.09 (30)	5.5	1.1	0.972	0.984	0.312
	0.95	100	1.09 (30)	5.5	1.1	0.972	0.986	0.314
	0.98 (default)	100	1.09 (30)	5.5	1.1	0.972	0.986	0.315
	0.99	100	1.09 (30)	5.5	1.1	0.972	0.987	0.316
	1.0	100	1.09 (30)	5.5	1.1	0.972	0.987	0.319
Event weight w_e	0.25	100	9.14 (194)	49.5	0.0	1.000	0.989	0.317
	0.5 (default)	100	1.09 (30)	5.5	1.1	0.972	0.986	0.315
	0.75	100	0.66 (18)	3.5	10.5	0.776	0.972	0.189
	1.0	100	0.42 (18)	1.5	26.2	0.568	0.932	0.117
Sensor weight w_s	0.0	100	0.66 (18)	3.5	36.5	0.482	0.824	0.217
	0.05	100	1.23 (30)	6	3.5	0.917	0.978	0.248
	0.15 (default)	100	1.09 (30)	5.5	1.1	0.972	0.986	0.315
	0.3	100	0.84 (30)	2	0.9	0.978	0.988	0.181
	0.5	100	0.49 (18)	2	0.9	0.978	0.988	0.149
BLOCK threshold	0.35	100	1.12 (30)	5.5	0.2	0.994	0.987	0.194
	0.4	100	1.09 (30)	5.5	0.2	0.994	0.987	0.216
	0.45 (default)	100	1.09 (30)	5.5	1.1	0.972	0.986	0.315
	0.5	100	1.09 (30)	5.5	1.1	0.972	0.986	0.323
	0.55	100	1.07 (30)	5.5	1.1	0.972	0.986	0.333

Decay factor λ. Surprisingly, λ between 0.90 and 1.00 (no decay at all) changed none of the detection or false-positive figures. In our scenarios trust moves mainly through large, sudden pieces of evidence, and these happen on a much shorter time scale than the decay half-life. The decay mostly affects devices that are *not* under suspicion. Mean honest trust rises slightly from 0.984 at $\lambda = 0.90$ to 0.987 without decay. Decay becomes important for recovery after a long quiet period (Figure 3.4) and would matter more against

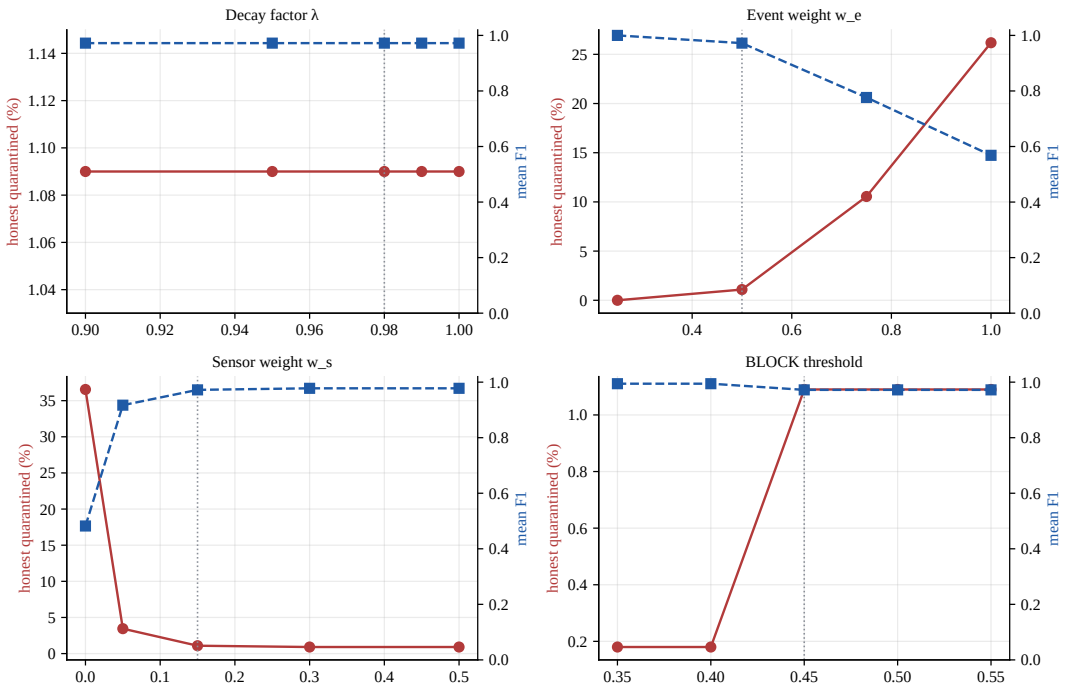


Figure 4.6: Sensitivity of honest quarantines (red, left axis) and mean F1 (blue, right axis) to four parameters. Dotted lines mark the defaults.

attackers that spread their moves further apart than our slow-and-low agent does.

Event weight w_e. This parameter controls the central trade-off of the engine. With $w_e = 0.25$, no honest device was ever quarantined, but evidence accumulated so slowly that the slow-and-low attacker needed a median of 49.5 ticks, and one attacker 194 ticks, before quarantine. With $w_e = 1$ (plain Bayes), the attackers were caught faster, but 26.2% of honest devices were quarantined, because a single maintenance access or reboot was enough to isolate a device. The default of 0.5 sits near the point where false quarantines are rare and detection is still fast.

Sensor weight w_s. Without sensors ($w_s = 0$) 36.5% of honest devices were quarantined, which confirms the ablation of Section 4.5 with a different number of seeds. Even a small weight of 0.05 brought this down sharply, and from 0.15 upwards it levelled off at about 1%, with faster detection as the weight grows. The silent-sensor terms act as a cushion of positive evidence on every clean request, so stronger sensors help both measures at once. This holds only because our simulated sensors are accurate and independent. With correlated or noisy real sensors, a high sensor weight would be risky.

BLOCK threshold. Lowering the BLOCK threshold to 0.40 or 0.35 reduced honest quarantines from 1.1% to 0.2%, with no loss of detection and almost no change in

detection time. Raising it to 0.55 changed little. The reason is the step-up challenge. An attacker that reaches the Suspicious band usually fails the multi-factor check, and the step-up failure term alone pushes it far below any of these thresholds. An honest device near the boundary, in contrast, usually passes the check and recovers. Our default of 0.45 is therefore slightly conservative for these scenarios. We kept it because a lower threshold leaves less margin against an attacker that can pass the second factor.

4.7 BAYESIAN VERSUS LLM WORLD (STAND-IN)

Tables 4.5 and 4.6 compare the two worlds scenario by scenario over 10 seeds (B = Bayesian, L = LLM world with the rule-based LLM stand-in). Figure 4.7 plots three of the parameters with their standard deviations.

Table 4.5: Twin-world comparison, part 1 (mean over 10 seeds; L = the rule-based LLM stand-in).

Scenario	**Detection rate (%)**		**Per-request accuracy (%)**		**Attacker requests allowed**	
	B	**L**	**B**	**L**	**B**	**L**
Normal day	—	—	99.2	94.5	0.0	0.0
Compromised (valid token)	100	10	99.5	95.4	1.7	13.3
Credential replay	100	100	99.2	95.4	1.7	1.8
Scanning	100	100	99.4	96.7	1.9	12.5
Token flood	100	100	99.2	94.8	4.3	44.2
Slow-and-low	100	0	98.0	86.1	21.9	195.9
Multi-attacker	100	70	99.7	97.0	5.6	71.0

Table 4.6: Twin-world comparison, part 2 (mean over 10 seeds; L = the rule-based LLM stand-in).

Scenario	**Legitimate requests blocked**		**Brier score**		**Trust separation gap**	
	B	**L**	**B**	**L**	**B**	**L**
Normal day	12.5	9.1	0.001	0.013	—	—
Compromised (valid token)	7.0	6.3	0.009	0.045	0.718	0.293
Credential replay	13.3	9.8	0.009	0.021	0.721	0.644
Scanning	10.2	6.8	0.010	0.021	0.704	0.574
Token flood	12.3	8.6	0.012	0.030	0.642	0.491
Slow-and-low	26.1	15.2	0.021	0.085	0.585	0.050
Multi-attacker	4.3	4.1	0.027	0.063	0.688	0.461

Averaged over the scenarios, the Bayesian world took the right action on 99.2% of requests against 94.3% for the LLM world, and its trust was better calibrated (mean Brier score 0.013 against 0.040). The two worlds agreed on 88.5% of requests. The largest differences are in the attacks that need several weak signals to be combined: the compromised device and the slow-and-low attacker, which the stand-in caught in only 10% and 0% of

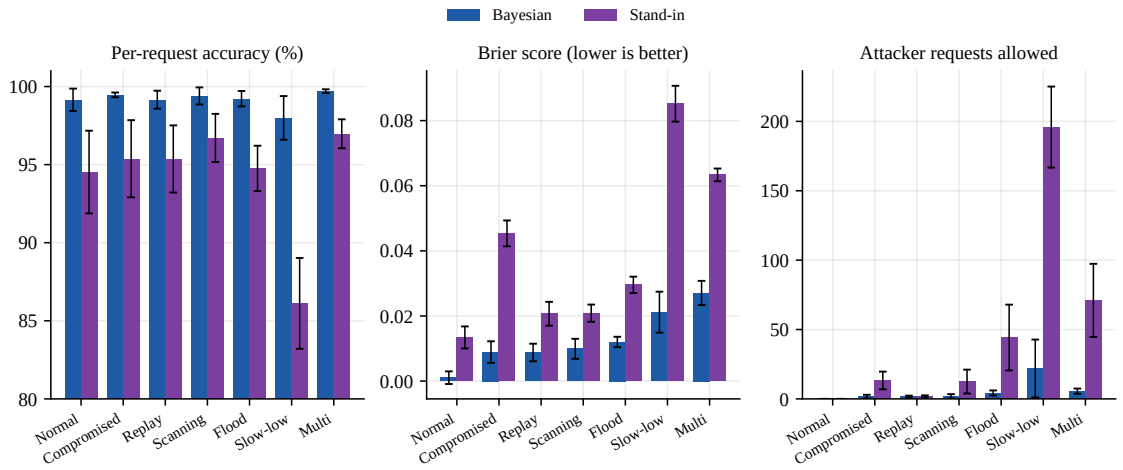


Figure 4.7: Per-request accuracy, Brier score and attacker requests allowed, per scenario (mean ± standard deviation over 10 seeds; LLM world = the rule-based LLM stand-in).

runs. In the other direction, the LLM world blocked slightly fewer legitimate requests, because it never quarantined an honest device in these runs, while the Bayesian engine occasionally quarantined a noisy-honest one. This was not a sign of gentler behaviour. A STEP-UP verdict stays in force until the device's next evaluation, so the LLM world issued tens to hundreds of step-up challenges per run against a handful in the Bayesian world, and it still let many more attacker requests through.

4.8 TRUST CALIBRATION

A trust value is meant to be a probability that the device is legitimate. A useful check is therefore whether devices that are given trust 0.9 really are legitimate about 90% of the time. We ran both worlds for 5 seeds of every scenario, grouped every device-tick by its trust value, and compared the mean trust in each group with the share of device-ticks that were truly legitimate (Table 4.7 and Figure 4.8). PENDING device-ticks in the LLM world have no trust value and are left out.

The Bayesian engine is strongly *separating* rather than finely calibrated. 89.3% of all device-ticks sit in the top bin (trust 0.9–1.0), and 99.8% of those were truly legitimate. In the four lowest bins (trust below 0.4) only 0.9% of the 3,246 device-ticks were legitimate; the rest belonged to active attackers. The bins between 0.5 and 0.8 are almost empty, because trust moves through them within a single request: a device is either clearly trusted or clearly isolated. The stand-in is coarser still. By construction its trust takes only four

Table 4.7: Reliability of trust values (all scenarios, 5 seeds; LLM world = the rule-based LLM stand-in).

	Bayesian			Stand-in		
Trust bin	**device-ticks**	**mean trust**	**legitimate (%)**	**device-ticks**	**mean trust**	**legitimate (%)**
0.0–0.1	763	0.042	0.0	199	0.050	0.0
0.1–0.2	802	0.144	0.0	0	—	—
0.2–0.3	1,151	0.258	2.6	0	—	—
0.3–0.4	530	0.353	0.0	2,770	0.320	0.0
0.4–0.5	2,387	0.447	6.3	0	—	—
0.5–0.6	1	0.508	0.0	0	—	—
0.6–0.7	8	0.612	12.5	4,183	0.620	61.0
0.7–0.8	29	0.748	96.6	0	—	—
0.8–0.9	359	0.829	96.7	0	—	—
0.9–1.0	50,220	0.989	99.8	46,451	0.920	97.9

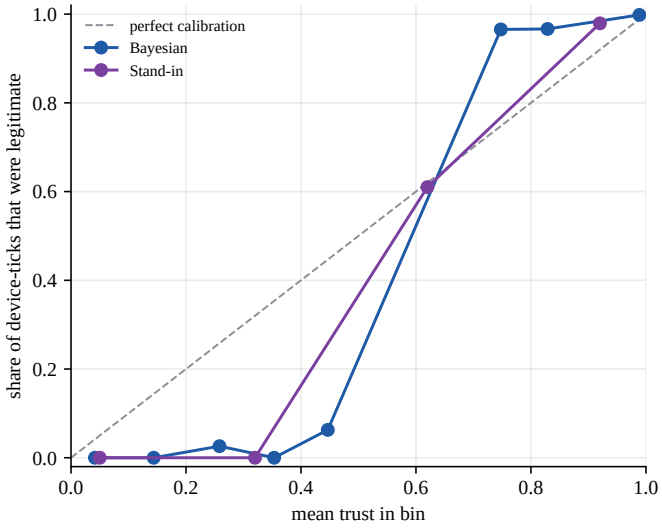


Figure 4.8: Reliability diagram: mean trust in each bin against the observed share of legitimate device-ticks (bins with at least 20 device-ticks).

values (0.92, 0.62, 0.32 and 0.05), and the 0.62 value, which means STEP-UP, was given to many device-ticks that were in fact legitimate. Both properties suit a zero-trust controller, which must act decisively, but they also mean the trust value should be read as a confident ranking rather than an exact probability. This is a direct consequence of the hand-set likelihoods and weights, which were chosen for clear decisions rather than fitted to data. The Brier scores in Table 4.6 summarise the same comparison in a single number per scenario.

4.9 COMPARISON BY ATTACK BEHAVIOUR

The comparison can also be organised by *what the attacker does*. The project infers each attacker's kind from observed facts only, using the behaviour signatures in Table 4.8. Ground truth is used only to score whether the inference was right.

Table 4.8: Behaviour signatures used to recognise each attack kind from observed facts.

Attack kind	**Behaviour signature (observed facts only)**
Compromised (valid token)	ZK passes, but ≥ 8 suspicious moves in one window (no link + other zone, unlinked sensitive target, or payload > $4\times$ baseline)
Credential replay	a reused nonce, or ≥ 2 ZK failures that no reboot explains (a reboot explains failures until the next valid proof on ordinary traffic)
Scanning	> 5 distinct ports in one window (honest devices use a single port)
Slow-and-low	ZK passes; ≥ 4 suspicious moves spread out (< 8 in any window), at least one with an exfiltration-sized payload
Token flood	≥ 15 token requests in one 20-tick window and a window rate $\geq 3\times$ its own baseline

Table 4.9 and Figure 4.9 compare the two worlds per attack kind, and Table 4.10 lists the evidence each engine reacted to when it detected the attacker. The signatures recognised every kind correctly except slow-and-low (80%). Its moves are spread out by design, so in some runs too few of them fall in the observed window.

Table 4.9: Detection by attack kind (attacker runs across all scenarios, 10 seeds; LLM world = the rule-based LLM stand-in).

Attack kind	**Runs**	**Recognised (%)**	**Detection (%)**		**Median ticks to BLOCK**	
			Bayesian	**Stand-in**	**Bayesian**	**Stand-in**
Compromised (valid token)	20	100	100	10	0	18.5
Credential replay	10	100	100	100	0	1
Scanning	20	100	100	100	0	1.5
Slow-and-low	10	80	100	0	5.5	—
Token flood	20	100	100	100	0	7.5

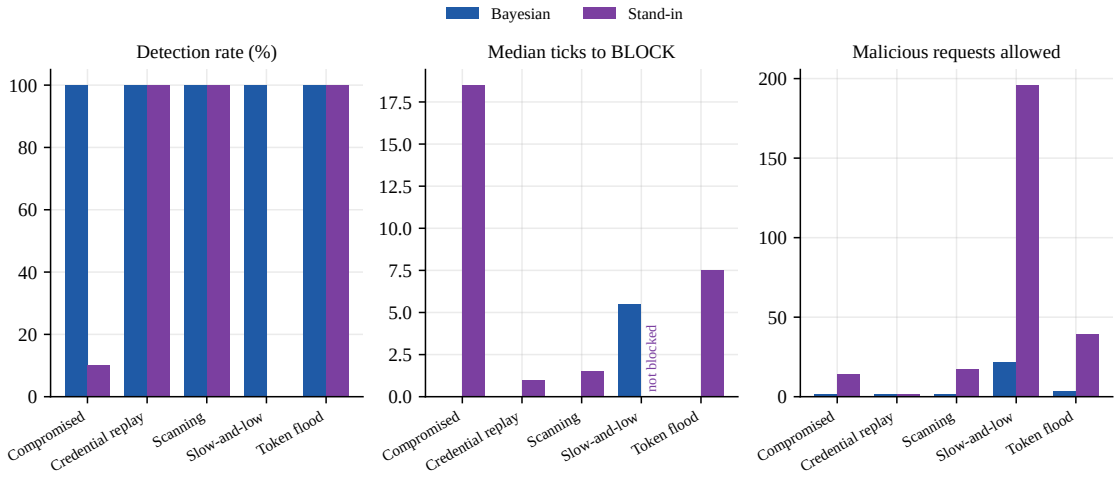


Figure 4.9: Detection rate, median ticks to BLOCK and malicious requests allowed, per attack kind.

Table 4.10: Evidence each engine reacted to when it detected the attacker (number of runs in brackets).

Attack kind	**Bayesian engine reacted to**	**Stand-in reacted to**
Compromised (valid token)	Sensor alarms (IDS / SIEM / BM) (20); Request with no configured link (20); Sensitive target (20)	Request with no configured link (20); Request rate far above baseline (2)
Credential replay	Step-up challenge result (10); Failed ZK proof (10); Sensor alarms (IDS / SIEM / BM) (10)	Reused nonce (10); Failed ZK proof (10); Many token requests (2)
Scanning	Step-up challenge result (20); Request with no configured link (20); Sensor alarms (IDS / SIEM / BM) (20)	Many destinations / ports (20); Request with no configured link (20)
Slow-and-low	Step-up challenge result (10); Reaching another zone (10); Request with no configured link (10)	Request with no configured link (10)
Token flood	Step-up challenge result (20); Sensor alarms (IDS / SIEM / BM) (20); Many token requests (13)	Request rate far above baseline (20); Many token requests (20)

4.10 CASE STUDY: ONE MULTI-ATTACKER RUN

Averages can hide how the two worlds actually behave, so Table 4.11 follows every device of a single run: the multi-attacker scenario with seed 0. In this run the camera starts scanning at $t = 35$, the smart meter is compromised at $t = 60$ and the thermostat starts flooding at $t = 90$.

Table 4.11: Per-device comparison for the multi-attacker scenario, seed 0 (B = Bayesian, L = the rule-based LLM stand-in).

		Final trust		First BLOCK (tick)		State changes	
Device	**Truth (attack start)**	**B**	**L**	**B**	**L**	**B**	**L**
DEV-001 Temperature Sensor	honest	0.99	0.92	—	—	0	1
DEV-002 Smart Camera	scanning (35)	0.14	0.32	35	36	1	13
DEV-003 Gateway Controller	honest	0.99	0.92	—	—	0	1
DEV-004 Industrial Actuator	honest	0.99	0.92	—	—	0	1
DEV-006 Smart Lock	honest	0.99	0.92	—	—	0	1
DEV-007 Smart Thermostat	token flood (90)	0.45	0.32	90	95	1	8
DEV-008 Smart Meter	compromised valid (60)	0.01	0.62	61	—	1	2
DEV-009 Infusion Pump	honest	0.99	0.92	—	—	0	1
DEV-010 Edge Gateway	noisy honest	0.99	0.62	—	—	0	4

In the Bayesian world the scanner and the flooder are blocked on the very tick their attacks begin, and the compromised meter one tick later. All three end the run at or below the BLOCK threshold. Every honest device, including the noisy-honest edge gateway, keeps trust near 0.99 and never changes state. The LLM world blocks the scanner one tick later and the flooder five ticks later, but it never blocks the compromised meter. The meter stays in the STEP-UP state, so each of its requests is challenged, but it is never quarantined. The LLM world's honest devices end at the stand-in's fixed "all clear" value of 0.92, and their single state change is the move out of PENDING after the first evaluation. The noisy-honest gateway, however, changes state four times and ends the run in STEP-UP, which shows how the stand-in turns harmless maintenance into repeated challenges. These are exactly the patterns that the averages in Tables 4.5 and 4.6 summarise.

4.11 COST, CONSISTENCY AND EXPLAINABILITY

The two engines differ greatly in cost. The Bayesian engine updates trust on every request, about 1557 updates per run, at a mean of 0.046 ms each on our machine. The LLM world made only about 203 evaluations per run, because it evaluates a device every 10 requests or after a hard event. Yet even a fast local model needs a fraction of a second

to a few seconds per call, so the LLM world is orders of magnitude slower in wall-clock time. For the same reason, about 75.1 device-ticks per run had no LLM trust value at all (PENDING), and devices changed state more often in the LLM world (2.41 state changes per device against 0.21).

Both worlds were perfectly consistent across three repeats of the same seed (100% and 100%). For the Bayesian engine this is guaranteed by construction. For the LLM world it holds only because the stand-in is deterministic. A real LLM, even at temperature 0, can return different answers on identical input, and the repeat measurement exists to quantify exactly that. The stand-in made 0 errors, and 100% of its stated factors were supported by facts in its window. With a real model, both numbers become meaningful measurements of reliability and of hallucination [9].

4.12 DISCUSSION

Our experiments lead to one main insight: *the quality of a zero-trust decision depends less on how clever the trust engine is than on whether it combines many weak, noisy signals consistently and on every request.* In this section we discuss each type of attack and then the overall picture.

4.12.1 Credential Replay and Token Flooding

These attacks leave unmistakable traces. A replayed nonce has a likelihood ratio of 0.005 against 0.5, so a single replay is decisive, and the cryptographic hard gate blocks the request in both worlds anyway. Token floods produce a rate spike and excess token requests within a few ticks. Both engines detected these attacks in every run. The Bayesian engine was faster because it re-evaluates on every request, while the LLM world must wait for its next scheduled evaluation.

4.12.2 Scanning

A scan creates many distinct ports and destinations, almost all of them without a configured link. The Bayesian engine adds these terms immediately and quarantined scanners on the first tick. The LLM world also caught every scanner, a tick or two later, letting between ten and twenty probes through in the meantime.

4.12.3 Compromised Device with a Valid Token

This is the case that motivated GAZETA [6], and it is where the engines differed most. The attacker's proofs are always valid, so only its behaviour can betray it. In the Bayesian world, each malicious move adds several moderate negative terms (no link, cross-zone, sensitive target, sensor alarms), and they accumulate within one tick to trigger a step-up, which the attacker then fails. The stand-in's coarse rules reacted only to "more than two requests without a configured link", which by itself gives STEP-UP rather than BLOCK, so it quarantined the attacker in only 10% of runs. A real LLM might weigh the zone and payload facts better. This is precisely the question the twin-world setup is designed to answer with a real model.

4.12.4 Slow-and-Low Attacker

The stealthy attacker is hard for both engines. Over 30 seeds, the Bayesian engine needed a median of 8.5 ticks and up to 64 to quarantine it. In the twin-world runs it let through 21.9 malicious requests on average before quarantine, while the LLM world with the stand-in never quarantined it and allowed 195.9. Spreading attacks out in time defeats any engine that looks only at a short window. An engine that never forgets would catch it, but would then also penalise honest devices for old, harmless anomalies. The decay factor λ controls this trade-off directly.

4.12.5 False Positives and Noisy Devices

Noisy-honest devices are the price of vigilance, and the two worlds paid it differently. The Bayesian engine challenged few of them (8% of noisy-honest devices were ever stepped up, averaged over scenarios in the twin-world runs) and occasionally quarantined one when a maintenance access and a failed MFA came together. The LLM world quarantined none, but stepped up 67% of them, and kept challenging every request until the next evaluation. In the Bayesian world the step-up is what keeps false alarms cheap. A successful step-up adds $+0.93$ to the log-odds and quickly restores trust, so an honest device usually pays one extra proof and one MFA prompt rather than 30 ticks of isolation. The ablation, however, reminds us that this only works because sensor silence on clean requests keeps honest trust high.

4.12.6 Overall Analysis

Across all scenarios, the transparent Bayesian engine was fast, accurate and fully explainable: every decision can be written out term by term, as in Table 3.6, and checked by hand. Its weaknesses are the ones we built into it: hand-chosen likelihoods, an optimistic sensor model and a fixed window. The LLM world shows the structural costs of an LLM-based engine, whatever the model: periodic rather than per-request evaluation, PENDING time before the first verdict, a higher wall-clock cost, and, with a real model, possible non-determinism and invented factors. Whether a real LLM's broader reasoning can outweigh these costs, especially for the compromised and slow-and-low attackers, cannot be answered by the stand-in. ZT-IoT Sentinel provides a fair, reproducible way to find out.

4.12.7 How the Results Meet Our Objectives

Table 4.12 links each objective from Section 1.4 to the evidence in this report.

Table 4.12: Objectives and the evidence that they were met.

Objective	**Evidence**
1. Reproducible zero-trust IoT testbed	Seeded random streams; identical results on repeated runs (consistency 100%); a test that the controller cannot see ground truth.
2. Authenticate every request without revealing secrets	Nonce-bound Schnorr proofs on every request; replayed proofs blocked by the hard gate in both worlds (Section 3.3).
3. Continuous, transparent trust estimation	Log-odds engine whose every update re-adds exactly (Table 3.6); all 240 attacker runs detected (Table 4.1).
4. Graded, reversible decisions	Step-up, 30-tick quarantine and probation; only 0.6–1.7% of honest devices quarantined.
5. LLM as an independent trust engine	Facts-only context, strict validation, no Bayesian fallback; local (Ollama), Gemini and OpenAI-compatible providers (Section 3.6).
6. Fair comparison and honest measurement	Isolated twin worlds, common metrics, ablation (Section 4.5), sensitivity analysis and calibration study.

4.12.8 Threats to Validity

We want to be clear about what these results do and do not show.

Internal validity. Every likelihood, threshold and rate in Table 3.11 is a modelling assumption, not a value measured from real IoT traffic. The sensors are driven by ground

truth and treated as independent. The ablation and the sensitivity analysis show how much the false-positive rate depends on these choices. The same team designed both the attacks and the evidence the controller looks for, which naturally favours detection.

External validity. The network has nine devices, the traffic is synthetic, and apart from slow-and-low the attacks are blatant by design: several malicious moves per tick. Real attackers adapt, real devices speak many protocols, and real sensors fail in correlated ways. Our numbers should not be read as the detection rate of a deployed system.

Construct validity. Detection time is measured in ticks, not seconds. A device counts as detected when it is quarantined, and the per-request truth is whether the sender was an active attacker at that tick. Other reasonable definitions, for example counting a step-up as a detection, would give different numbers. For this reason we report both.

The LLM side. The LLM-world numbers in this chapter come from a rule-based stand-in, not from a language model. They demonstrate the comparison method and the structural costs of periodic evaluation, but not the quality of any real model.

Cryptography. The zero-knowledge parameters are teaching-sized and not secure. They demonstrate the protocol, not its security level.

Chapter 5

CONCLUSION

IoT devices are easy to compromise, and a perimeter network that trusts a device once it has logged in gives an attacker with a stolen credential a free path across the network. Zero trust answers this by authenticating every request and re-estimating every device's trust from its behaviour. This project set out to make that idea concrete, measurable and easy to inspect, and to ask whether a large language model could take over the job of the trust engine.

We built **ZT-IoT Sentinel**, a zero-trust controller for a simulated IoT network inspired by GAZETA [6]. Every request carries a nonce-bound Schnorr-style zero-knowledge proof, so a device never reveals its secret and a replayed proof is rejected at once. A Bayesian engine then updates the device's trust in log-odds space from evidence the controller derives itself, such as missing links, cross-zone access, scans, rate spikes, unusual payloads and replayed nonces, and from three imperfect sensors. Tempering weights and a slow decay keep that trust stable. A policy engine with hysteresis turns trust into ALLOW, STEP-UP or BLOCK, and step-up challenges, time-limited quarantine and probation keep false alarms cheap. Beside it, we integrated a large language model, local through Ollama or in the cloud through Google Gemini, as a completely independent trust engine. It sees only observed facts, must answer in a strict JSON schema, and is never allowed to fall back to Bayesian values. Running the two engines in isolated twin worlds on identical seeded traffic lets us compare them request by request against ground truth.

Over 210 seeded runs, the Bayesian engine quarantined the attacker in all 240 attacker runs, usually on the very tick the attack began, while quarantining only 0.6–1.7% of honest devices. The stealthy slow-and-low attacker was the hardest to catch, with a median of 8.5 and a maximum of 64 ticks. Our ablation study showed that much of the low false-

positive rate comes from our optimistic sensor model: without sensor evidence, attackers were still caught but honest quarantines rose to 16–50%. The twin-world comparison, validated with a clearly labelled rule-based stand-in, showed that the pipeline works end to end. It also showed the structural costs any LLM-based engine must overcome: periodic rather than per-request evaluation, a PENDING period before the first verdict, and a much higher cost per decision.

In short, this work shows that a transparent Bayesian trust engine, whose every decision can be written out and checked by hand, is a strong and explainable baseline for zero-trust IoT security, and it provides a fair, reproducible testbed in which any LLM-based alternative can be measured against it.

5.1 FUTURE WORK

Several directions follow naturally from this project:

1. **Richer models of IoT behaviour.** Today every device in our simulator speaks a simplified request language: a destination, a request kind, a port and a payload size. Real IoT systems behave in far more varied ways, and a future version could simulate them in much more detail:

    - *protocol-level behaviour*: MQTT publish and subscribe on specific topics, CoAP request and observe patterns, BLE pairing and advertising, LoRaWAN uplinks limited by duty cycles, and Modbus reads and writes to industrial registers;
    - *device life-cycle behaviour*: over-the-air firmware updates, key rotation, battery-saving sleep cycles and planned maintenance windows;
    - *physical-process behaviour*: sensor readings that follow a real process, such as the temperature of a room or the flow rate of an infusion pump, so that an attacker who injects impossible values can be noticed;
    - *protocol-specific attacks*: topic hijacking, malformed packets, rogue gateways and protocol downgrades.

    Each richer behaviour would give the controller new, more specific evidence, for example “this meter published to a topic it has never used” or “this pump reported a

flow rate outside its physical range". It would also make the honest devices noisier and therefore make false positives harder to avoid. Testing both trust engines against such behaviour would bring the simulator much closer to a real IoT deployment.

2. **Game-theoretic policies.** Replace our fixed thresholds with authentication policies derived from the equilibrium of a GAZETA-style game [6, 18], and compare their cost and robustness with the threshold policy.

3. **Realistic evidence.** Drive the sensors from real IDS output or public IoT traffic datasets instead of ground truth, and learn the likelihood table from data rather than setting it by hand.

4. **Smarter attackers.** Add adaptive attackers that react to step-up challenges and quarantine, and longer slow-and-low campaigns, to stress the decay and window settings.

5. **Hybrid engines.** Let the LLM explain or audit the Bayesian engine's decisions rather than replace it, combining the speed and auditability of the Bayesian engine with the language model's ability to describe what it sees.

6. **Deployment.** Replace the teaching-size zero-knowledge parameters with a standard elliptic-curve group and run the controller against real devices on a small testbed.

REFERENCES

[1] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proceedings of the 26th USENIX Security Symposium*, pp. 1093–1110, 2017.

[2] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[3] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, "Demystifying IoT security: An exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2702–2733, 2019.

[4] V. Hassija, V. Chamola, V. Saxena, D. Jain, P. Goyal, and B. Sikdar, "A survey on IoT security: Application areas, security threats, and solution architectures," *IEEE Access*, vol. 7, pp. 82721–82743, 2019.

[5] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," tech. rep., The MITRE Corporation, 2018.

[6] Y. Ge and Q. Zhu, "GAZETA: GAme-Theoretic ZEro-Trust Authentication for Defense Against Lateral Movement in 5G IoT Networks," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 540–554, 2024. doi: 10.1109/TIFS.2023.3326975.

[7] H. Xu *et al.*, "Large language models for cyber security: A systematic literature review." arXiv preprint arXiv:2405.04760, 2024.

[8] M. A. Ferrag *et al.*, "Revolutionizing cyber threat detection with large language models." arXiv preprint arXiv:2306.14263, 2023.

[9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[10] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, p. 20, 2019.

[11] J. Kindervag, "No more chewy centers: Introducing the zero trust model of information security," tech. rep., Forrester Research, 2010.

[12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[13] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.

[14] A. Jøsang and R. Ismail, "The beta reputation system," in *Proceedings of the 15th Bled Electronic Commerce Conference*, 2002.

[15] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[16] Z. Yan, P. Zhang, and A. V. Vasilakos, "A survey on trust management for Internet of Things," *Journal of Network and Computer Applications*, vol. 42, pp. 120–134, 2014.

[17] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead," *Computer Networks*, vol. 76, pp. 146–164, 2015.

[18] Y. Ge and Q. Zhu, "Trust threshold policy for explainable and adaptive zero-trust defense in enterprise networks," in *Proceedings of the IEEE Conference on Communications and Network Security (CNS)*, 2022.

[19] Y. Ge and Q. Zhu, "Zero trust for cyber resilience." arXiv preprint arXiv:2312.02882, 2023.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[21] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[22] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[23] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[24] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology — CRYPTO '86, Lecture Notes in Computer Science, vol. 263*, pp. 186–194, Springer, 1987.

[25] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.

[26] A. Grattafiori *et al.*, "The Llama 3 herd of models." arXiv preprint arXiv:2407.21783, 2024.

[27] Gemini Team, Google, "Gemini: A family of highly capable multimodal models." arXiv preprint arXiv:2312.11805, 2023.

[28] Q. Zhu and S. Rass, "On multi-phase and multi-stage game-theoretic modeling of advanced persistent threats," *IEEE Access*, vol. 6, pp. 13958–13971, 2018.

[29] E. Bertino and N. Islam, "Botnets and Internet of Things security," *Computer*, vol. 50, no. 2, pp. 76–79, 2017.

[30] B. Bowman, C. Laprade, Y. Ji, and H. H. Huang, "Detecting lateral movement in enterprise computer networks with unsupervised graph AI," in *Proceedings of the 23rd*

International Symposium on Research in Attacks, Intrusions and Defenses (RAID), pp. 257–268, 2020.

[31] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.